



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

School of Computer Science and Statistics

Creating Novel Virtual Reality Interfaces for Teaching Computer Architecture

Priyansh Nayak

B.Sc. in Computer Science, University of Arizona

A Dissertation submitted in partial fulfilment
of the requirements for the degree of

Master of Science in Computer Science
(Augmented and Virtual Reality)

Supervisor: Gareth W. Young

August 2026

Declaration

I, the undersigned, declare that this work has not previously been submitted as an exercise for a degree at this, or any other University, and that unless otherwise stated, is my own work.

Priyansh Nayak

August 14, 2026

Permission to Lend and/or Copy

I, the undersigned, agree that Trinity College Library may lend or copy this thesis upon request.

Priyansh Nayak

August 14, 2026

Creating Novel Virtual Reality Interfaces for Teaching Computer Architecture

Priyansh Nayak, Master of Science in Computer Science

University of Dublin, Trinity College, 2026

Supervisor: Gareth W. Young

This dissertation investigates whether a bounded virtual reality learning environment can support students' understanding of selected single-cycle MIPS datapath concepts as a supplementary educational tool. Although standard diagrams and verbal explanations can introduce instruction execution, learners may still struggle to form a coherent mental model of how values, control choices, and architectural state changes relate across the datapath. In response, the dissertation presents a VR prototype that translates instruction execution into a guided spatial walkthrough organised around six learner-facing phases: Fetch, Decode, Execute, Memory, Write-Back, and Program Counter Update. Designed as a pedagogical abstraction rather than a cycle-accurate simulator, the prototype supports selected instructions across Learning mode, Practice mode, and Test mode.

The prototype was evaluated through an exploratory participant study using questionnaire responses, a post-session knowledge check, a limited baseline/post-session comparison, open-ended feedback, and researcher observations. Results showed generally positive usability and perceived-learning responses alongside recurring discomfort and interface-friction issues. Knowledge-check performance was generally strong, while the repeated comparison showed stronger descriptive performance on some items following the overall learning session. Because participants received a tailored refresher before entering the headset, these differences cannot be attributed to VR alone. Researcher observations showed some progression into reduced-support use alongside continuing difficulties with controls, information presentation, and selected lesson decisions.

The findings suggest that the prototype shows promise as a supplementary means of making selected computer architecture concepts more tangible and easier to follow, while remaining limited by the exploratory study design, participant comfort, and interface readability. The dissertation's main contribution is the development and evaluation of a pedagogical translation of single-cycle datapath reasoning into a sequential, learner-facing VR experience.

Acknowledgments

I owe this project first and foremost to my beloved parents, to whom I owe everything. Their love, more than I could ever repay, support from halfway across the world, and willingness to give me the opportunity to pursue this degree made all of this possible.

I thank my advisor, who helped pull me from one of the darkest periods of my life and set me on the path that eventually led me to this course and to Trinity.

I thank the friends I made along the way, who had my back throughout the academic year and helped keep me sane through the madness of the semesters. We pulled one another through difficult stretches, and even their quiet presence beside me while we worked made the long days feel less lonely.

I thank my supervisor, Gareth, for trusting me with this project, giving me the freedom to pursue it, and guiding me throughout a journey in which I was repeatedly given the chance to prove to myself what I was capable of.

More than anything, I thank the person I was a year and a half ago: the one who had grown accustomed to the dark, who was content to remain in the shadow of others, and who had begun to accept that this was simply where he belonged. He was afraid of taking even the first step forward, but he took it anyway. Somewhere along the road that followed, through the people around me and the responsibilities I was forced to grow into, I learned how to lead again. That first step is now so far behind me that I can no longer see where it began.

Teaching has always held a special place in my life. I once thought that to teach was simply to be a guiding light for someone else: to share what you know and help them find their way. Experience taught me that it is also an exchange. Every person you teach brings with them a perspective that changes your own, and in helping someone else understand, you are often made to understand anew yourself.

To teach is also to blaze a trail far enough that those who follow no longer need your light to find their way. If something of what you gave them remains, they carry those embers onward, beyond where you could have taken them yourself, leaving their own marks among a night already full of stars. That belief has stayed with me through every year I have spent teaching and learning, and, in many ways, it is the reason this dissertation became the project that it did.

PRIYANSH NAYAK

University of Dublin, Trinity College
August 2026

Contents

Abstract	iii
Acknowledgments	iv
Chapter 1 Introduction	1
1.1 Research Question and Scope	2
1.2 Reader’s Guide	2
Chapter 2 Background and Related Work	4
2.1 Computer Architecture as a Learning Problem	4
2.2 Conventional Teaching and Its Limits	6
2.3 Interactive Support for Computer Architecture Learning	7
2.4 Virtual Reality in Computer Science Education	8
2.5 Design Constraints and Transferable Lessons	8
2.6 Implications for Design	9
2.7 Summary	10
Chapter 3 Methodology	12
3.1 Research Framing	12
3.1.1 Research Question	12
3.1.2 Evaluation Aspects	13
3.2 Alignment with the Learning Outcomes	13
3.3 Study Design and Participants	14
3.4 Procedure	14
3.5 Instruments and Data Capture	15
3.6 Analysis Plan	16
3.7 Ethical Considerations and Study Boundaries	17
3.8 Use of Generative Artificial Intelligence	18
3.9 Summary	18

Chapter 4	Design	19
4.1	Learning Outcomes and Design Scope	19
4.2	Pedagogical Position	20
4.2.1	From Ordinary Teaching to Supplementary Support	20
4.2.2	Guidance, Progression, and Independence	21
4.3	Representational Strategy and Abstraction	21
4.3.1	Why Abstraction Was Necessary	21
4.3.2	Preserving Reasoning Rather Than Literal Timing	22
4.3.3	Selected Representational Choices	23
4.4	Environment Structure and Lesson Flow	23
4.4.1	The World-Space Route	23
4.4.2	The Six Lesson Phases	25
4.5	Modes, Interaction, and Local Support	25
4.5.1	Modes as Variations on One Lesson	25
4.5.2	Physical Action and Local Guidance	26
4.5.3	Validation and Conceptual Discipline	26
4.6	Core Design Decisions and Trade-Offs	27
4.7	Summary	27
Chapter 5	Implementation	28
5.1	Platform and Development Context	29
5.2	Overall System Architecture	29
5.3	Environment Structure and Lesson Routing	30
5.3.1	Scene Organisation	30
5.3.2	Guidance, Gates, and Local Interfaces	30
5.4	Representation of Architectural State	32
5.4.1	Instruction Representation	32
5.4.2	Register Representation	33
5.4.3	Datapackets and Value Flow	33
5.4.4	Scanner Representation	34
5.4.5	Control Signals	35
5.4.6	Memory Representation	36
5.5	Implementation of the Lesson Phases	36
5.5.1	Instruction Fetch	37
5.5.2	Decode and Operand Preparation	37
5.5.3	Execute	38
5.5.4	Memory Access	39
5.5.5	Write-Back	40

5.5.6	Program Counter Update	41
5.6	Implementation of the Three Lesson Modes	43
5.6.1	Learning Mode	43
5.6.2	Practice Mode	43
5.6.3	Test Mode	43
5.7	Implemented Content Baseline	44
5.7.1	Supported Instruction Set	44
5.7.2	Authored Register and Memory Baseline	45
5.8	Technical Constraints and Deliberate Simplifications	45
5.9	Summary	47
Chapter 6	Results	48
6.1	Participant Background and Study Context	48
6.2	System-Experience Results	50
6.2.1	Usability	51
6.2.2	Discomfort	51
6.2.3	Engagement	52
6.2.4	Perceived Learning Experience	53
6.3	Knowledge-Check Results	54
6.4	Baseline and Post-Session Comparison	55
6.5	Qualitative Results	58
6.5.1	Open-Ended Questionnaire Feedback	58
6.5.2	Researcher Observations and Lesson Progression	59
6.6	Summary	61
Chapter 7	Discussion	62
7.1	Usability, Clarity, and Interface Friction	62
7.2	Conceptual Support and Knowledge Outcomes	65
7.3	Progression from Guided to Reduced Support	67
7.4	Value as a Supplementary Educational Tool	69
7.5	Study and Prototype Limitations	70
7.6	Answer to the Research Question	71
7.7	Summary	72
Chapter 8	Conclusion	73
8.1	Main Contribution and Research Question	73
8.2	Future Work	74
	Bibliography	76

Chapter A Participant Questionnaire	80
Chapter B Participant Information Leaflet	86
Chapter C Informed Consent Form	89
Chapter D Recruitment Materials	91

List of Tables

2.1	Comparison of key reviewed literature	11
5.1	Learning-mode instruction set	44
5.2	Practice-mode instruction set	45
5.3	Authored register baseline	46
5.4	Authored memory baseline	46
6.1	Participant background summary	49
6.2	Questionnaire descriptive summary	50
6.3	Post-session knowledge summary	55
6.4	Baseline/post-session comparison summary	56
6.5	Repeated-item baseline/post transitions	57
6.6	Summary of recorded lesson progression and support	60
6.7	Recorded instruction use across lesson modes	60

List of Figures

2.1	Reference single-cycle datapath overview	5
4.1	Datapath-to-VR translation overview	22
4.2	Routed lesson map	24
4.3	Three-mode lesson progression	26
5.1	Overview of the VR lesson environment	28
5.2	Guidance cues along the route	31
5.3	Three-panel local interface split	31
5.4	Instruction-module states across fetch progression	32
5.5	Physical register-bank representation	33
5.6	Datapackets for immediate extension and ALU output	34
5.7	Scanner states across validation	35
5.8	Visible data-memory structure	36
5.9	Instruction-module docking during fetch	37
5.10	Reduced-support decode in Practice mode	38
5.11	ALU validation during execute	39
5.12	Memory-access interaction in Practice mode	40
5.13	Write-back as a visible culmination	41
5.14	Program-counter update and branch consequence	42
5.15	Mode selection at lesson entry	42
6.1	Usability-item response distribution	51
6.2	Discomfort-item response distribution	52
6.3	Engagement-item response distribution	52
6.4	Learning-experience response distribution	53
6.5	Post-session knowledge percent correct	54
6.6	Repeated-item baseline/post comparison	57

Chapter 1

Introduction

Computer architecture is the study of how a computer is organised and how its components work together during the execution of a program. In an undergraduate computer science degree, it usually appears only after students have already spent time with programming, discrete mathematics, and smaller hardware abstractions, establishing a foundation before they begin looking more closely at how software is carried out by the machine [1–4]. What makes the subject distinctive is that it asks students to connect those earlier experiences with the behaviour of the machine itself, moving from writing programs to reasoning about registers, control, memory, and instruction flow.

It is also where the subject often starts to feel harder than what came before. Students are no longer working only with source code or with isolated components, but with instruction fields, registers, arithmetic logic unit (ALU) behaviour, memory access, control decisions, and program counter behaviour that all have to make sense together. Traditional teaching builds toward this carefully, and for good reason: lectures, whiteboard derivations, traces, diagrams, and simulators all contribute something useful. Even so, many students reach a point where they can name the parts of a datapath and still cannot clearly explain what is happening when a particular instruction runs. The problem is less about missing facts than about learning to see execution as a connected process rather than as a labelled picture.

Studies of architecture teaching report the same pattern from different angles. Porter et al. found that students in computer architecture courses often underperformed instructor expectations on supposedly basic concepts [5], while Yehezkel et al. argue that learners can otherwise end up treating the machine too much like a black box rather than forming a viable mental model of its behaviour [6]. Tan et al. write that the subject is often perceived as “dry, tough, and complex” and that students struggle because they are “unable to visualize these multiple-step procedures” [7]; earlier work likewise describes architecture learning as difficult and, at times, unmotivating under conventional presentation [8].

This dissertation grew out of seeing that teaching problem repeatedly during undergraduate teaching-assistant work in computer organisation, where students could often identify the parts of a datapath but still struggled to explain how a particular instruction moved through the processor as a connected process. In those moments, the most useful intervention was often not another verbal explanation but a slower walkthrough that drew over the datapath, followed the instruction step by step, and made each decision visible. The project presented here takes that teaching pattern as its starting point and asks whether a virtual reality (VR) environment can serve as a useful supplement at precisely that point of difficulty.

The prototype is framed as a supplement to lectures, diagrams, and conventional simulators rather than as a cycle-accurate machine model. Its focus is a selected set of single-cycle Microprocessor without Interlocked Pipeline Stages (MIPS) instructions and a bounded set of learning tasks: decoding the instruction, selecting the correct operands, following the path of execution, making control-related choices, and reasoning about the resulting architectural state. To support different stages of learning, the environment is organised around three modes: Learning mode, Practice mode, and Test mode. These gradually reduce guidance while keeping the same overall lesson structure.

1.1 Research Question and Scope

Within that bounded scope, the dissertation is guided by the following question:

To what extent can an immersive virtual reality learning environment support students' understanding of selected single-cycle MIPS datapath concepts as a supplementary educational tool?

The dissertation asks whether this particular prototype, used within its intended educational scope, provides meaningful support for understanding selected single-cycle MIPS datapath concepts and whether participant evidence justifies treating it as a promising supplementary learning tool. The evaluation is therefore framed as exploratory rather than as a controlled test of whether virtual reality is superior to conventional teaching. The study is consequently concerned with conceptual support and guided tracing within that supplementary role.

1.2 Reader's Guide

The remainder of the dissertation is organised as follows. Chapter 2 places the project in context by reviewing the background and related work, with particular attention

to computer architecture learning, virtual reality in computer science education, and the practical limits of existing teaching support for first-pass mental-model building. Chapter 3 then sets out the methodological frame for the dissertation, including the research question, evaluation aspects, learning outcomes, study design, instruments, analysis plan, and ethical boundaries.

Chapter 4 explains the design of the learning environment itself, showing how those aims were translated into pedagogical choices, environment structure, interaction design, and the main abstractions used to make datapath behaviour teachable in VR. Chapter 5 describes the implementation of the prototype. Chapter 6 presents the study findings, Chapter 7 discusses their meaning, and Chapter 8 returns to the research question, summarises the main contributions, and identifies directions for future work.

Chapter 2

Background and Related Work

This chapter places the project in context. It begins with the educational difficulty posed by computer architecture, then considers what conventional teaching already does well and where some learners still struggle, before turning to interactive and immersive work in computer architecture and computer science education. It closes by identifying the design constraints and narrower gap from which the current prototype emerged.

2.1 Computer Architecture as a Learning Problem

Computer architecture asks students to reason about a machine whose most important behaviour is hidden. They must understand how an instruction is fetched, how its fields are decoded, how register values are selected, how the ALU is used for different instruction classes, how memory is addressed, and how the program counter is updated. None of this is beyond explanation on paper, but the subject places a particular burden on the learner because several representations have to be held together at once. A datapath diagram shows structure, an instruction format shows encoding, an assembly statement shows intent, and a trace shows outcome. The difficulty lies in turning those partial views into one connected account of execution.

Porter et al. developed concept questions for computer architecture courses and found that students were often “not learning as much as the instructors expected,” with a per-question average of only 56% across the studied cohorts [5]. Their account is especially useful because it separates procedural success from conceptual understanding. Students may learn how to complete familiar exercises and still, in their phrase, miss “the proverbial forest for the trees” [5]. In architecture teaching, that distinction is important. A learner may be able to label datapath components, fill in a pipeline table, or work through a cache example while still struggling to explain what the processor is doing at a given moment.

Work on architecture visualisation points in much the same direction. Yehezkel et al. argue that students using conventional tools can end up treating the machine too much like a black box, whereas a stronger visual environment can help them construct “a viable mental model of the computer” [6]. The problem, then, is not simply that architecture contains difficult facts. The harder task is to connect assembly syntax, instruction fields, datapath structure, register state, memory state, and control behaviour over time into a single explanation of execution.

The work reported here concentrates on instruction execution in a single-cycle MIPS datapath, a limited but still revealing part of this wider area. A learner still has to decode the instruction, identify source and destination registers, select operands, follow value flow through the datapath, and predict the resulting architectural state.

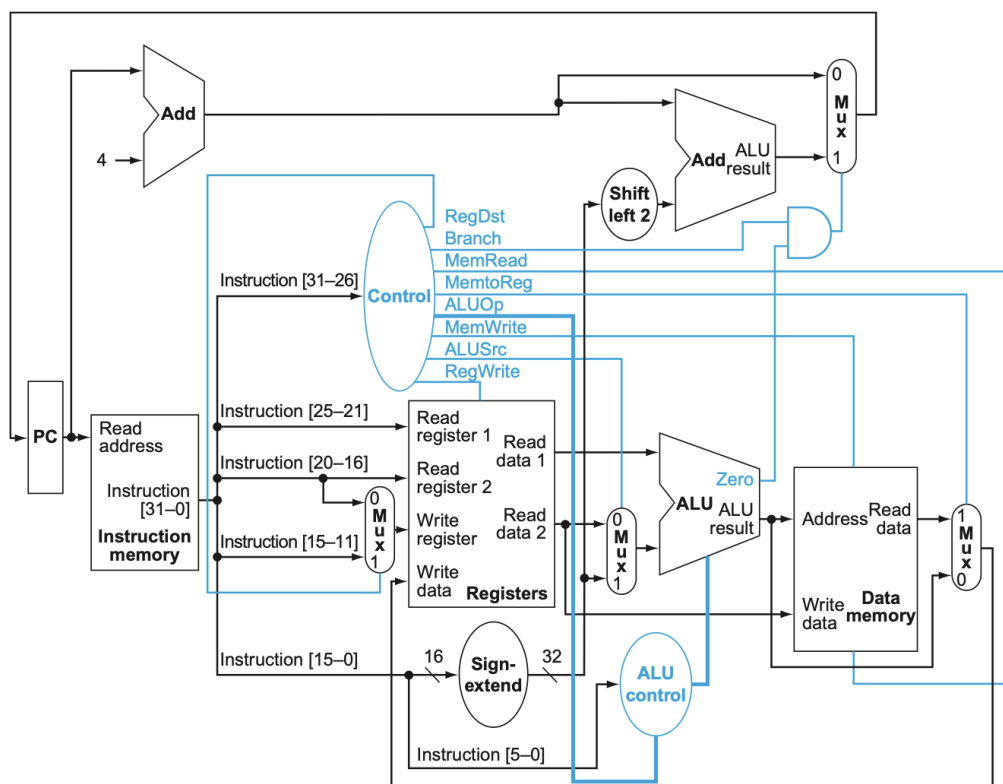


Figure 2.1: Reference single-cycle MIPS datapath used to establish the conventional processor representation discussed throughout the dissertation, attributed to Jyotiprakash Mishra’s blog post on the topic [9]. The diagram provides the structural baseline against which the later VR translation of instruction flow, register access, ALU behaviour, memory access, and program-counter updates is compared.

2.2 Conventional Teaching and Its Limits

None of this implies that conventional teaching is ineffective. In most programmes, computer architecture is taught progressively rather than all at once. Basic logic comes first, then gates, then combinations such as half adders and full adders, then larger datapath components, and only after that does the learner meet the processor as an integrated system. Official module descriptions reflect that build-up directly through sequences that move from digital logic and register transfer into datapath design and the fetch-decode-execute cycle [2]. That staged development matters, and nothing in this dissertation argues against it.

Traditional tools also do several things very well. Lectures and tutorials provide order. Whiteboard derivations slow a problem down and show how one step follows from the next. Static datapath diagrams are strong at showing structure, naming components, and fixing the shape of the machine in the learner’s mind. Simulators and assembly tools are valuable for correctness, inspection, and repeated practice because they let students test programs, inspect registers and memory, and connect assembly syntax to machine behaviour in a more formal way than a verbal explanation alone.

A learner cannot make sensible use of the VR environment without some prior exposure to the underlying architectural ideas. The prototype was never intended to replace the gradual conceptual build-up provided by lectures, diagrams, exercises, and conventional tools. It sits alongside those materials at the point where a student has already seen the structure of the datapath but still struggles to turn that structure into a working mental model of execution flow.

Fukuda et al. frame a related problem in systems teaching as a balance between “extreme abstract” learning and “extreme technical” learning [10], a distinction that is useful here because it suggests that observation alone may leave students with too little experience and too little comprehensive understanding. The same distinction helps define the present problem. Conventional teaching can still leave some learners needing additional help at the point where observed behaviour has to be turned into a connected explanation.

The limitation of conventional teaching is therefore narrower than a claim that the tools do not work. Some of the most useful traditional materials are strongest either at showing structure or at checking correctness. They may be less helpful during first-pass mental-model building for learners who need to see how information moves through the machine as a sequence of linked decisions. A datapath diagram can show where the ALU sits, and a simulator can show that a register changed, but for some students the connection between those two facts is not immediate. A spatial tool is attractive precisely in that gap.

2.3 Interactive Support for Computer Architecture Learning

Before turning to VR, it is worth noting that Tlili et al. had already explored architecture learning through another interactive format, using an educational mobile game in which learners found the approach motivating and were willing to use it again [8]. Even without immersion, the study still matters because it suggests that the subject responds well to interactive presentation and because it keeps attention on the deeper issue of making abstract system behaviour easier to follow.

More directly related work appears in immersive and experiential systems. Dascalu et al. describe an early VR environment for studying computer architecture through experiential learning, in which learners assembled virtual computer components while receiving textual guidance [11]. The emphasis differs substantially from the current project, since their interaction centres on assembling computer components rather than tracing instruction execution through an established datapath. Nevertheless, the work provides a useful early precedent for incorporating direct physical manipulation into computer-architecture learning and for moving the subject away from passive explanation toward structured activity inside a virtual environment.

The closest architecture-focused precedent reviewed here is the VR serious game described by Tan et al. [7], whose authors describe Computer Organisation and Architecture as “dry, tough, and complex” and argue that students struggle because they cannot “visualize the procedures of various CPU functions” [7]. Their system is organised around tutorials, self-guided assignments, and quizzes, and it covers topics such as fetching and executing instructions, interrupt handling, interrupt-driven input/output, and context switches. Help material is provided in a “step-by-step manner” before that support is reduced in later tasks [7]. This provides a relevant precedent for the present project, particularly in the use of structured guidance followed by more independent activity. The similarity is primarily pedagogical as rather than adopting the same interaction model, the prototype developed here places greater emphasis on spatial movement through a routed datapath environment and direct physical interaction with learner-facing representations of instruction execution. Its scope is also narrowed more tightly around single-cycle datapath execution.

Checa et al. provide a nearby precedent from computer-hardware learning rather than architecture proper [12]. Their VR serious game is about hardware assembly, not datapath tracing, but the study suggests that immersive learning may be more helpful for understanding, visual recognition, and making connections between concepts than for simple factual recall [12]. That distinction matters here. The aim is not primarily to help students memorise terminology, but to help them connect instruction fields,

datapath structure, and resulting machine state into one explanation.

Alnuaimi and Awad’s recent VR environment for digital design and computer organisation laboratory learning is also relevant for a simpler reason: it keeps usability in view throughout the evaluation [13]. Even a promising educational VR system has to be learnable and comfortable enough to use before any learning claim can matter. A tool intended to support understanding of hidden processes can just as easily create another layer of confusion if the interface is awkward or overloaded.

2.4 Virtual Reality in Computer Science Education

The broader computer science education literature gives a clearer sense of where architecture sits within immersive teaching work. Reviews of VR in computer science education note its promise in areas where abstract concepts benefit from interactive visualisation and show that many applications are directed toward conceptual learning rather than toward full practical programming tasks [14–16]. Janani et al. are especially useful because they treat VR as strongest for abstract and process-oriented topics while also describing it as a complementary teaching medium [16].

Pirker et al. report stronger subjective measures such as presence, absorption, and positive emotion when comparing VR and desktop learning for sorting algorithms [17]. A mobile VR application for learning Python also reports encouraging results, but frames VR more as a useful aid than as a complete substitute for ordinary teaching [18]. Dengel reaches a similarly cautious conclusion from another angle, arguing that immersive environments can support learning in computer science without implying that higher immersion will automatically yield better learning outcomes in every case [19]. Across these studies, the same pattern appears: VR is often most plausible as a way of making a first encounter with a difficult idea more concrete.

The same reviews also point to the limits of the field. Recurring usability issues, weak collaborative features, and a lack of clear instructional grounding appear across parts of the literature, and the work remains scattered across topics and methods [15, 16]. A useful contribution, then, does not depend on claiming that VR in computer science education is entirely new; it can instead take the form of a careful application in a domain where the match between problem and medium is especially strong.

2.5 Design Constraints and Transferable Lessons

A case for VR still has to be carried by the quality of the design itself. The harder question is what kind of VR design is likely to help rather than hinder. Cook et al. discuss practical constraints in educational VR, drawing attention to issues such as

deployment, accessibility, sustainability, and the amount of support institutions need before VR becomes a normal teaching tool [20]. The emphasis therefore shifts away from spectacle and toward a more modest question: can a system be clear, usable, and worth the time required to use it?

Korkut and Surer similarly show that immersive space does not automatically solve information-design problems [21]. Poor layout, visual clutter, and badly organised interaction can still make a system hard to read. That lesson carries directly into architecture teaching, where the central challenge is already one of representing a hidden process without losing the learner in detail: if too much of the datapath is surfaced at once, VR may simply reproduce the original problem in another form.

A broader review of VR in education reaches a similar conclusion by showing continuing interest in the medium alongside repeated concerns about readiness, hardware access, discomfort, and the quality of actual learning design [22]. Lee et al. make a different but related argument: immersive experience can help restore intuitive understanding where abstraction has flattened away the underlying reality [23]. Although their work is not about architecture, the idea transfers well: a datapath diagram is useful precisely because it abstracts, yet that abstraction can become too detached from the felt sequence of execution. The central design problem is therefore not how to make the learner wander through a visually impressive machine, but how to make the essential steps of execution more graspable without adding another layer of clutter.

2.6 Implications for Design

The literature points toward a fairly specific use of VR in computer architecture. It also suggests that the most useful systems are selective in what they externalise, what they keep simple, and where they ask the learner to act rather than only observe.

Architecture learning also appears to benefit from guidance and progression. Tlili et al. and Tan et al. both use structured support before learners are expected to work more independently [7, 8]. Similar patterns appear in the broader computer science education literature, where prompts, panels, and gradual reduction of support are common design choices [16]. Taken together, these approaches informed the decision to begin with stronger guidance and gradually require more independent reasoning as the learner progressed.

The literature also warns repeatedly against overload. A datapath can be made more visible in VR, but that does not mean every hidden detail should be turned into an explicit interaction. If the learner is asked to manage too many objects, too much text, or too much movement at once, much of the value of the medium quickly disappears [20–23]. These concerns informed the decision to make only selected parts

of the datapath directly interactive, rather than turning every underlying relationship into a separate learner task.

Against that background, the gap addressed here can be stated fairly plainly. There is already evidence that interactive and immersive approaches can support computer architecture learning, and there is a broader body of work showing that VR can help with abstract computer science concepts. Less well served is a tightly scoped learning environment centred on manual instruction tracing through a single-cycle datapath, with explicit attention to operand selection, stage-by-stage decisions, and the final architectural state. That is the space occupied by the current prototype. The next chapter turns from the literature to the method used to evaluate it.

2.7 Summary

The literature supports a narrower claim than any simple assertion that VR is automatically good for education, or automatically good for computer architecture. Existing work shows that architecture learning has already been explored through mobile games, experiential environments, and VR serious games, while the wider VR-in-computer-science-education literature suggests that conceptual learning is one of the medium's more plausible uses. Table 2.1 provides a compact comparison of the key interactive, immersive, and design-oriented works reviewed in this chapter, showing how the literature progresses from game-based architecture support toward more immersive and more explicitly constrained educational VR systems. As that comparison makes clear, the same literature also sets clear limits: guidance matters, usability matters, and information has to be presented with restraint. Those conditions shape the prototype evaluated in the next chapter.

Study	Medium	Focus	Support style	Relevance
Tlili et al. [8]	Mobile serious game	Computer architecture learning	Guided interactive tasks with feedback and increasing difficulty	Early interactive architecture-learning precedent.
Dascalu et al. [11]	Virtual reality environment	Experiential computer-architecture learning	Text-guided virtual component assembly	Early immersive architecture-learning precedent.
Tan et al. [7]	VR serious game	Computer organisation and architecture	Step-by-step tutorials followed by self-guided tasks and quizzes	Closest direct precedent for guided visualisation of processor procedures.
Checa et al. [12]	VR serious game	Computer hardware assembly	Guided tutorials followed by autonomous assembly	Adjacent evidence for immersive hands-on technical learning.
Alnuaimi and Awad [13]	VR laboratory	Digital design and computer organisation	Task-based virtual laboratory with usability evaluation	Near-domain precedent for usability, clarity, comfort, and learner readiness.
Janani et al. [16]	Scoping review	VR in computer science education	Concept-oriented and scaffolded immersive learning patterns	Broader positioning for VR as complementary conceptual support in computer science education.
Cook et al. [20]	Expert-led qualitative study	Educational VR deployment	Institutional design, accessibility, support, and sustainability concerns	Practical counterweight on deployability, access, and support burden.

Table 2.1: Comparison of selected interactive, immersive, and design-oriented literature reviewed in this chapter. The studies show the progression from game-based computer-architecture support to architecture-focused VR and broader educational-VR work, while highlighting the guidance, usability, and deployment considerations that informed the prototype.

Chapter 3

Methodology

This chapter sets out the method used to evaluate the prototype. The study was conceived as a small-scale exploratory evaluation of a deliberately scoped educational system; it was not designed as a controlled comparison or as a test of whether VR should replace conventional teaching.

3.1 Research Framing

The evaluation considered whether the prototype was educationally useful within its intended scope, whether participants could engage with it clearly enough for that value to matter, and whether the combined evidence justified further development for selected computer architecture concepts.

As noted in Chapter 1, the prototype grew out of repeated teaching-assistant experience in computer organisation. The study therefore examined whether the guided tracing that proved useful in that setting could be translated into a useful VR-based learning environment, a question that also sits in line with earlier architecture-focused immersive work [7, 11].

3.1.1 Research Question

The dissertation was guided by the following research question:

To what extent can an immersive virtual reality learning environment support students' understanding of selected single-cycle MIPS datapath concepts as a supplementary educational tool?

In this dissertation, the phrase “to what extent” is to be interpreted in a bounded exploratory sense rather than as a promise of a full classroom control-group comparison. More specifically, the question is answered through three linked forms of evidence:

participant perceptions of the environment, descriptive concept evidence drawn from the baseline subset and post-session responses, and observed progression or support needs across the session structure.

The purpose of the study was therefore to determine whether this particular prototype, used in this particular educational context, offered enough conceptual clarity, usability, and perceived value to justify the design choices behind it and support later work on broader evaluation.

3.1.2 Evaluation Aspects

The research question was examined through four linked evaluation aspects reflecting the prototype’s intended educational role:

1. **Usability and clarity.** Whether participants could understand, navigate, and use the system clearly enough for its educational purpose to be evaluated.
2. **Conceptual support.** Whether the collected evidence suggested that the environment helped participants follow instruction flow, datapath phases, component roles, and selected architectural relationships.
3. **Progression from guided to reduced support.** Whether participants could move from the guided structure of Learning mode into less-supported interaction in Practice or Test mode, and what support remained necessary during that transition.
4. **Supplementary educational value.** How participants perceived the prototype in relation to lectures, diagrams, tutorials, simulators, and other conventional teaching materials.

3.2 Alignment with the Learning Outcomes

The evaluation was aligned with the learning outcomes defined for the prototype, but those outcomes belong primarily to the design of the system and are discussed in full in Section 4.1. In methodological terms, they matter here because they establish what the study should look for and what kinds of claims the evidence can reasonably support.

The study therefore focused on a bounded cluster of educational tasks: decoding supported instructions, explaining the roles of major datapath components, tracing instruction and value flow across the main lesson phases, making the required operand and control-facing choices, predicting resulting architectural state changes, reasoning across different supported instruction classes, and completing the walkthrough with

progressively less support. These categories were reflected across the baseline subset, the post-session knowledge check, the observation notes, and the broader interpretation of participant feedback.

3.3 Study Design and Participants

Structured evidence came from the background form, the baseline subset, the Likert-style post-session items addressing perceptions of the system, and the knowledge check on selected concepts. Open-ended evidence came from written participant feedback and researcher observation notes recorded during and after the session. These sources were considered together to provide a fuller account of how the prototype functioned as a learning tool.

The evaluation remained exploratory rather than classroom-trial in scale. There was no formal non-VR control group and the study did not attempt to measure long-term retention. The design instead focused on how participants engaged with the prototype and whether the collected evidence supported its intended role as a supplementary learning environment, consistent with broader literature on conceptual VR support in computer science education [16, 19].

Twelve participants took part in the completed study through voluntary convenience sampling. All participants were consenting adults aged 18 or over. The approved ethics application had initially set a target sample size of twenty participants, while the completed recruitment yielded twelve. Recruitment took place by email invitation, no payment was offered, and the study involved no dependent relationship between the research team and participants. Background information on prior architecture exposure, prior VR use, confidence, and topic familiarity was collected before the session so that the findings could be interpreted in light of participants' different starting points; these characteristics are reported in Chapter 6.

3.4 Procedure

Each participant session followed the same broad structure: consent and background capture, a short baseline probe, a tailored refresher, the VR session itself, and a post-session evaluation.

The session began with the Participant Information Leaflet and written informed consent, both of which are reproduced in Appendix B and Appendix C respectively. Participants were given the opportunity to read the study information, ask questions, and decide whether they wished to continue before any data collection began. They then completed the background section of the questionnaire reproduced in Appendix A,

which captured prior VR use, prior exposure to computer architecture or computer organisation, and self-reported familiarity and confidence.

After the background section, participants were offered a short baseline subset drawn from the knowledge check reproduced in Appendix A. This consisted of five pre-session questions used as a modest checkpoint on foundational ideas before the VR experience began, specifically Questions 11–14 and 25 from the post-session questionnaire in Appendix A. Because this baseline acted as a quick pre-session scan rather than a mandatory scored gate, unanswered items were retained as missing rather than treated as incorrect. Participants then received a short verbal refresher tailored to their prior experience. The refresher was intentionally high-level: it was not meant to teach the whole content in advance, but to align terminology, reduce avoidable confusion, and give participants enough orientation to engage meaningfully with the prototype. Typical coverage included the basic form of an assembly instruction, what the datapath represents, what the main instruction stages do, how registers and memory differ, where control choices matter, and what kinds of interaction participants would encounter in the VR environment.

Participants then completed the VR session using selected instructions and one or more of the available modes. A recorded walkthrough of the prototype, representative of what participants were seeing during the live study, is available at <https://youtu.be/Y8tFYnWeLtI>. The exact path depended on participant background, session time, and whether the participant appeared comfortable enough to move into reduced-guidance modes. A typical path included one or two Learning mode instructions, one Practice mode instruction, and, where appropriate, a further Test mode attempt. During the session, the researcher intervened only when needed, and participants were encouraged to make full use of the resources available inside the environment. The amount and kind of intervention required were themselves treated as meaningful observational data.

After removing the headset, participants completed the remaining sections of the questionnaire reproduced in Appendix A. These included Likert-style ratings on usability, engagement, and perceived learning, followed by the broader knowledge-check section and open-ended feedback. Alongside the formal questionnaire data, the researcher maintained brief session notes on participant support, attempted instructions and modes, points of hesitation, and relevant comparisons with traditional materials.

3.5 Instruments and Data Capture

The primary structured instrument was the Microsoft Forms questionnaire used in the live study. It captured participant background, post-session usability, engagement

and perceived-learning responses, the targeted knowledge check and follow-up confidence item, and open-ended feedback. A reconstructed print version of the deployed questionnaire is reproduced in Appendix A, while the Participant Information Leaflet, consent form, and representative recruitment materials are reproduced in Appendix B, Appendix C, and Appendix D.

The knowledge-check portion of the live form focused mainly on topics relevant to the implemented prototype: program-counter behaviour, fetch, the instruction register, stage order, the execute stage, branch-related reasoning, and the behaviour of supported MIPS instructions such as `add`, `addi`, `lw`, `sw`, `beq`, and `j`. A small number of surrounding architecture concepts were also retained so that responses could be interpreted against the broader background knowledge with which participants entered the session, while the main evaluative focus remained on the selected datapath concepts supported by the prototype.

Some participant-facing materials, most explicitly the Participant Information Leaflet in Appendix B, described the study using broader computer architecture examples such as pipelining, memory hierarchy, and stack behaviour. These materials are reproduced unchanged for transparency because they formed part of the approved ethics documentation before the final prototype scope was fully settled. The evaluated VR session itself remained limited to the selected single-cycle MIPS datapath concepts defined in this dissertation.

Researcher notes supplemented the questionnaire by recording attempted instructions and modes, points of hesitation, support requirements, and relevant comparisons participants made with conventional learning materials.

3.6 Analysis Plan

The analysis was primarily descriptive and combined evidence on usability, conceptual support, participant understanding, and supplementary educational value. The small sample and exploratory design did not justify strong inferential testing, so the evidence was interpreted through descriptive statistics, item-level patterns, open-ended feedback, and researcher observations. IBM SPSS Statistics was used for quantitative analysis, while NVivo supported the organisation of qualitative material.

Quantitative analysis summarised participant background, item-level Likert responses and descriptive group measures, knowledge-check performance, post-knowledge-check confidence, and the limited baseline/post-session comparison. Frequencies, percentages, means, medians, ranges, and internal-consistency estimates were reported where appropriate to the measure being described. Baseline and post-session responses were compared descriptively only on the repeated items for which matched data were

available.

Open-ended responses and researcher observation notes were read together through a light thematic analysis. The analysis focused on recurring material concerning what helped understanding, what remained confusing, interface or navigation friction, the guidance structure, and comparisons with conventional learning materials.

Because the study did not include a formal non-VR control group, comparison remained indirect and drew on participant perceptions, the limited repeated baseline/post-session items, and observed progression from guided to less-guided performance. The baseline subset was followed by a short verbal refresher before the VR session itself, so any baseline/post-session differences were interpreted as performance following the overall learning session rather than as evidence attributable to the VR environment alone.

3.7 Ethical Considerations and Study Boundaries

The study received research ethics approval through the School of Computer Science and Statistics Research Ethics Management System (REAMS). Participation was voluntary and based on informed consent, with participants receiving study information before taking part and retaining the right to withdraw without penalty. They were also informed that withdrawal of research material might no longer be possible once data had been anonymised and incorporated into the analysis.

The principal participant risk was temporary discomfort associated with VR use, including eye strain, dizziness, or motion sickness. Participants could pause or stop the session immediately if they felt unwell, and task completion was not prioritised over participant comfort.

Administrative contact information was kept separate from research responses, and research materials were anonymised as early as practicable. Access to identifiable data was restricted to what was necessary for the study, with storage limited to password-protected and university-approved systems in line with the approved data-handling plan. Identifiable or coded administrative data were retained only for a limited period before deletion.

The study used a Meta VR headset supplied through the School, and participants were not required to sign into personal Meta accounts. The research team did not intentionally collect Meta account data, although participant materials noted that limited device-level or platform-level processing outside the direct control of the research team could still occur.

3.8 Use of Generative Artificial Intelligence

In accordance with Trinity College Dublin’s *College Statement on Artificial Intelligence and Generative AI in Teaching, Learning, Assessment & Research* (2024), the use of generative artificial intelligence in the development of this project and preparation of the dissertation is acknowledged below.

ChatGPT (GPT-5.6 Sol, OpenAI, <https://chatgpt.com>) was used in a supporting capacity to clarify selected computer-architecture concepts during development, including walkthroughs of how supported instructions operate through a single-cycle datapath. It was also used to help standardise BibTeX entries where citation information obtained from Google Scholar and individual publisher websites was inconsistent, and to provide critique and proofreading of author-written dissertation material.

Codex (GPT-5.4, OpenAI, <https://openai.com/codex/>) was used in a limited support capacity. Its primary use was to help maintain and organise a project journal so that implementation milestones and decisions could later be referred back to as historical summaries. It also provided limited assistance with repository organisation, the Python data-processing pipeline used to clean, restructure, and prepare collected study data for analysis, and the preparation of tables. Codex was not used to create, fabricate, invent, simulate, or alter participant data, results, or findings.

3.9 Summary

This chapter established the methodological framework for the exploratory evaluation, including the study design and participants, procedure, data-collection instruments, analysis strategy, and ethical considerations. The analysis remained primarily descriptive, while the tailored refresher between the baseline probe and VR session means that any baseline/post-session differences reflect the overall learning session rather than an effect attributable to VR alone. Chapter 4 turns from this methodological framework to the design decisions through which the targeted datapath concepts were translated into a guided, learner-facing VR environment.

Chapter 4

Design

The previous chapters established the teaching difficulty addressed by the project and the bounded way in which it was evaluated. This chapter explains how that difficulty was translated into a learning environment. The design problem was to decide which parts of instruction execution needed to become visible, which decisions should be left in the learner’s hands, which abstractions could clarify the process without distorting it beyond use, and how far the whole system could be narrowed while still addressing the original educational concern in a serious way.

As outlined in Chapter 2, computer architecture is ordinarily taught through a gradual build-up that begins with logic and simpler hardware ideas before moving toward datapaths, control, and instruction execution [2], and the prototype was therefore designed to support the later transition from recognition to explanation.

4.1 Learning Outcomes and Design Scope

The prototype was designed around a specific set of learning outcomes rather than around the broader and vaguer ambition of helping students “understand computer architecture.” Those outcomes were used to decide what the environment should teach, what kinds of learner action it had to preserve, and where the project had to stop. By completing the VR experience, learners should be able to

1. Decode the relevant fields of a supported single-cycle MIPS instruction;
2. Explain the roles of the major datapath components involved in instruction execution;
3. Trace an instruction and its data through the major stages of a single-cycle datapath;

4. Select the correct operands and control-facing choices needed to execute a supported instruction;
5. Predict the architectural state changes caused by that instruction;
6. Compare how different instruction classes change datapath behaviour; and
7. Complete a supported instruction walkthrough with reduced or no guidance.

These outcomes define an educational progression: first making sense of the instruction as an encoded object, then recognising the components and stages through which it travels, then reasoning through the choices that govern its execution, and finally carrying that reasoning forward with less support. The structure of the environment follows that sequence closely, which is why the lesson path, mode progression, and supported instruction set are all tied to the same small cluster of tasks.

The prototype does not claim to teach the whole of assembly programming, full digital-logic construction, pipelining, hazards, or the entire content of an undergraduate architecture course. It is also not intended to function as a comprehensive MIPS simulator or a general digital-logic tutor. It concentrates on selected single-cycle instruction walkthroughs because that is the point at which the educational problem can be made concrete without the system collapsing under its own breadth. A broader instruction set or curriculum target would have weakened the coherence of the build and blurred the claims that could reasonably be made about it.

A prototype built around only one clean example would have made the lesson easier to stage, but it would also have hidden the fact that different instruction classes activate the datapath differently and demand different forms of reasoning from the learner. The final scope therefore had to be broad enough to cover arithmetic, immediate, memory, branch, and limited jump behaviour, while remaining narrow enough for the lesson loop to stay legible and stable throughout the participant study.

4.2 Pedagogical Position

4.2.1 From Ordinary Teaching to Supplementary Support

The project was conceived from the beginning as a supplement to ordinary teaching rather than as an alternative to it. Lectures, tutorials, whiteboard derivations, simulators, and textbook diagrams already do several things extremely well: they provide sequence, terminology, worked examples, and a stable reference image of the machine. As Chapter 2 argued, the relevant difficulty appears only after that earlier exposure has already taken place. The prototype therefore assumes prior exposure to the topic

and takes its place after that exposure, where it can reinforce, clarify, and slow down the reasoning rather than trying to replace the earlier stages of instruction.

That role shaped the tone of the whole environment. The system was meant to provide a more guided and more tangible form of walkthrough at the stage where diagrams and explanations have already introduced the topic but have not yet settled into a stable mental model. The design is closer to guided tracing on a whiteboard than to an open simulation or unguided exploratory world.

4.2.2 Guidance, Progression, and Independence

The pedagogical model is built around supported progression. Learning mode slows the process down, supplies explanation at the relevant moment, and keeps the learner oriented as the instruction advances through the datapath. Practice mode asks for more independent decoding and more deliberate decision-making while preserving the same overall route. Test mode strips back much of the remaining support and asks whether the learner can now carry the reasoning through with less intervention. This structure follows a pattern already visible in related architecture and educational game work, where tutorials and strongly supported tasks precede stricter or more independent activities [7, 8].

A student who can complete a task only while being coached through every step has not reached the same point as a student who can reconstruct the same reasoning more independently; at the same time, a learner who is denied support too early may never build the underlying structure in the first place. The three-mode arrangement reflects that balance by letting support recede gradually rather than treating explanation and independence as mutually exclusive states.

4.3 Representational Strategy and Abstraction

4.3.1 Why Abstraction Was Necessary

The central representational problem arose from a mismatch between hardware behaviour and teachable experience. A textbook single-cycle datapath carries out its work as one overall cycle whose internal activity is largely hidden, combinational, and parallel. Fetch, decode, operand selection, ALU behaviour, memory access, write-back, and determination of next program-counter can be named separately for teaching purposes, yet they are not experienced by the hardware as six clear, sequential teaching moments of the kind a learner usually needs before the process becomes intelligible.

For that reason, the design could not simply imitate the diagram and hope that immersion alone would close the gap. If every internal relationship remained as

compact and as hidden as it is in the textbook picture, the environment would inherit much of the same first-pass difficulty that motivated the project in the first place. Work on VR in computer science education and immersive information design points in the same direction: the educational value of the medium depends on whether it clarifies the conceptual structure of the task without overwhelming the learner [16, 20, 21]. The abstractions used in the prototype were therefore justified by teachability before they were justified by realism.

4.3.2 Preserving Reasoning Rather Than Literal Timing

The guiding decision was to preserve the reasoning structure of instruction execution even when the literal timing and invisibility of the hardware had to be relaxed, which in practical terms meant treating the familiar phases of Fetch, Decode, Execute, Memory, Write-Back, and Program Counter Update as teaching phases rather than as claims about separate clocked steps in a real processor. The prototype thus remains grounded in the logic of a single-cycle datapath, but, as shown in Figure 4.1, it presents that logic in a form that can be followed, revisited, and acted upon by a human learner.

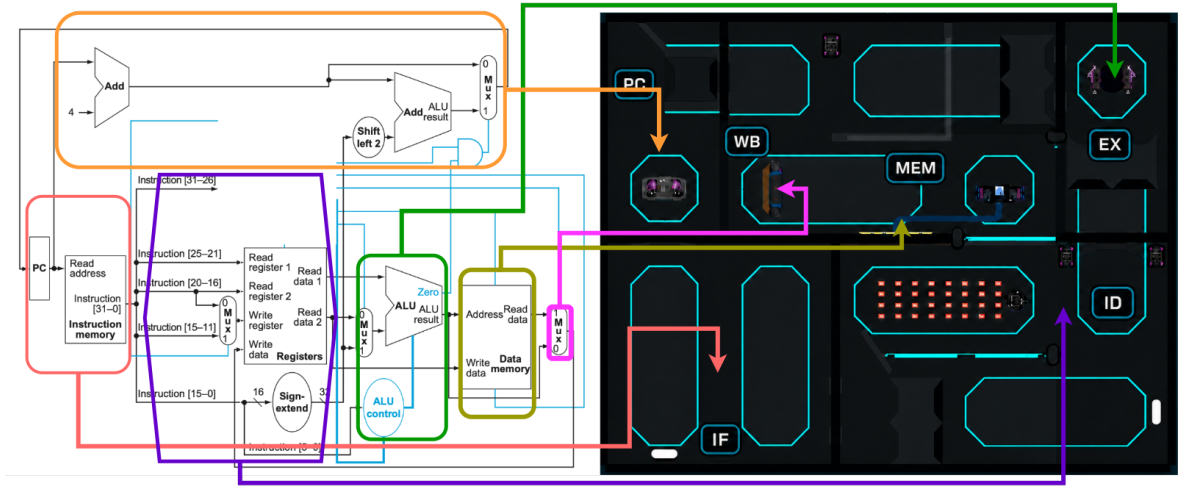


Figure 4.1: Overview of how the traditional single-cycle datapath was translated into the routed VR environment. The figure shows how major textbook datapath regions were reorganised into learner-facing stations so that instruction execution could be followed as a visible sequence without implying separate hardware clock stages.

Several of the prototype’s most visible abstractions follow from that choice. Fetch becomes a simpler visible acquisition and transfer event rather than a silent response to the program counter. Decode is given an explicit learner-facing role rather than being left as internal field interpretation and control generation. Memory use, write-back, and program-counter consequences each receive enough space to become

legible parts of the execution story. These changes retain the underlying datapath relationships while redistributing them into a form that the learner can follow sequentially.

4.3.3 Selected Representational Choices

The physical register bank and the later use of “datapackets” offer the clearest example of this strategy. During decode, the learner benefits from seeing source and destination registers as stable architectural locations that must be identified correctly. Once the lesson moves beyond that point, the more useful question concerns value flow: which quantity is moving, where it came from, and what later stage depends upon it. Datapackets answer that question more directly than a rigid insistence on keeping every value attached to a register-shaped object.

The same principle shaped the treatment of control logic and multiplexers, much of which is centralised and resolved internally in the textbook datapath. In the prototype, multiplexers are not modelled as separate learner-facing components, and some control-facing consequences are distributed across the later phases of the lesson. That choice keeps Execute, Memory, Write-Back, and Program Counter Update educationally active, whereas a stricter concentration of all control into Decode would have left those later phases looking more like consequences to observe than decisions to understand.

Some hidden relationships were worth externalising because doing so made the logic easier to follow; others were better left in local panels, prompts, or background system behaviour because turning them into further embodied tasks would have added friction without much educational return. The result is not a full simulator of every internal detail, but a narrower representational model shaped around what a learner most needs in order to reconstruct the flow of execution coherently.

4.4 Environment Structure and Lesson Flow

4.4.1 The World-Space Route

Guided tracing required the lesson to unfold as a route rather than as a single viewing surface. The prototype was organised as a sequence of world-space stations through which the learner follows the active instruction, allowing the environment to control the pace at which each part of execution is introduced.

As shown in Figure 4.2, the routed map, directional cues, gate logic, and local landmarks all serve that purpose. They help the learner keep track of where they are in the execution path, what has already been resolved, and what remains to be decided. Traditional datapath diagrams can be powerful as overview material, but they present

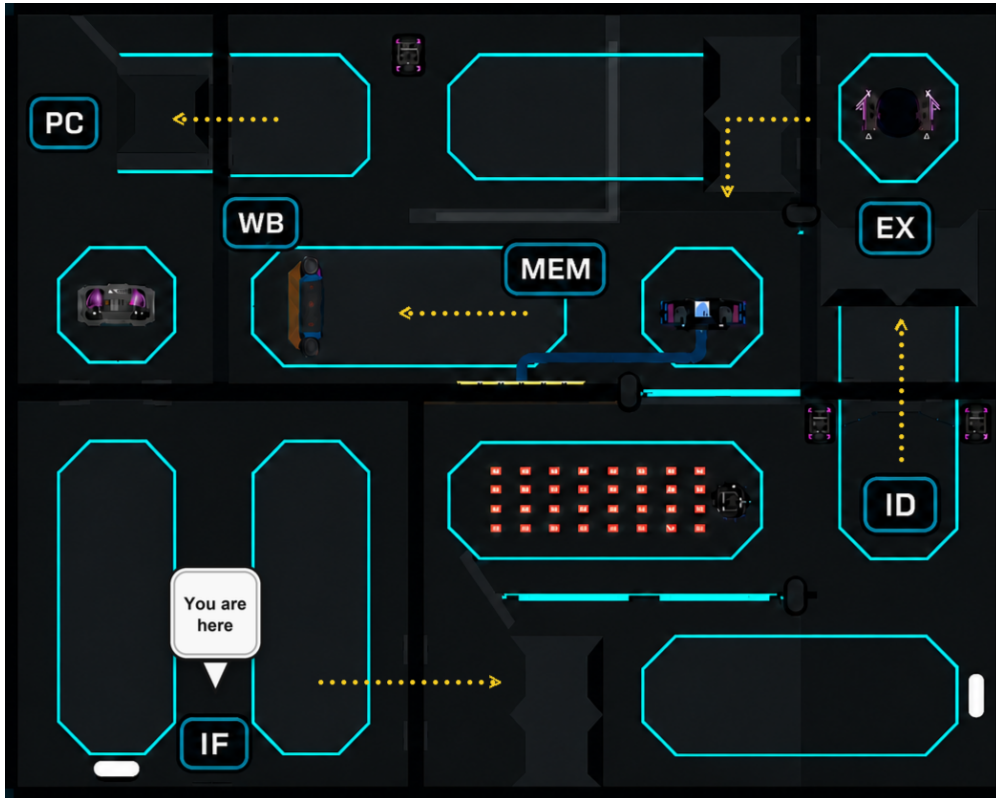


Figure 4.2: Annotated map of the VR environment showing the ordered route through the main lesson stations. The spatial sequence controls when each part of instruction execution becomes available to the learner and supports progression from one learner-facing phase to the next.

the entire system at once. In the prototype, the same system is encountered in a controlled sequence, which reduces the pressure to hold every relationship in mind at the same time.

4.4.2 The Six Lesson Phases

As Figure 4.2 shows, the route itself is organised around six familiar lesson phases, isolated as parts of the reasoning chain: Fetch, Decode, Execute, Memory, Write-Back, and Program Counter Update. Fetch introduces the active instruction and establishes that execution begins from a definite source. Decode turns encoded structure into usable information by requiring the learner to identify fields, source operands, and immediate-related meaning where relevant. Execute gives architectural operations a visible decision point rather than leaving the ALU as a silent internal region in the diagram. The Memory phase clarifies the distinction between arithmetic behaviour and memory behaviour by giving addressing and data access their own local context. Write-Back provides a clear moment at which the learner can connect previous decisions with the final architectural update. Program Counter Update is separated for similar reasons: control-flow consequences are often harder to absorb when they remain implicit, so the design gives them a dedicated place in the lesson sequence. Across all six phases, the learner is not being asked to memorise a route through a virtual world for its own sake, but to reconstruct the flow of instruction execution through a series of linked decisions whose order becomes easier to follow once they are made spatial and local.

4.5 Modes, Interaction, and Local Support

4.5.1 Modes as Variations on One Lesson

As outlined in Section 4.2.2, the prototype uses Learning mode, Practice mode, and Test mode as variations on one lesson structure rather than as separate interpretations of the datapath. What changes across the modes is the level of support, tolerance, and learner responsibility.

Learning mode provides the fullest local explanation and therefore carries the strongest teaching load, helping the learner build an initial model of how an instruction moves through the datapath. Practice mode reduces that support while preserving enough scaffolding for the learner to attempt the route more deliberately. Test mode retains the same structure under stricter conditions and with much less help. This continuity matters because it lets the learner revisit the same reasoning under changing levels of support rather than confronting a different lesson each time.

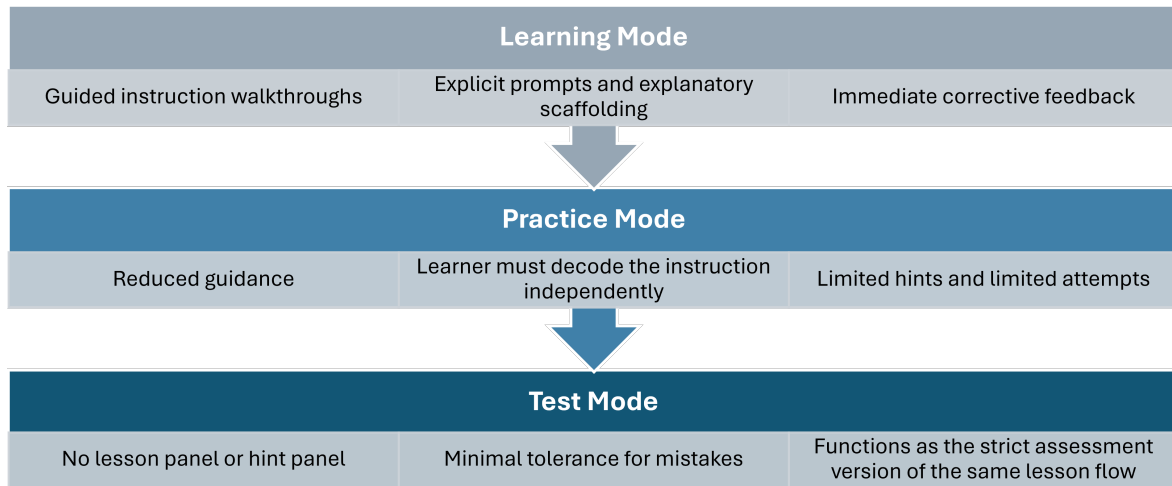


Figure 4.3: Pedagogical progression across Learning mode, Practice mode, and Test mode. The same lesson route is retained across all three modes while explanation, hints, tolerance, and other forms of support are progressively reduced so that the learner is asked to reconstruct more of the reasoning independently.

4.5.2 Physical Action and Local Guidance

The interaction model follows the same selective logic already described in the representational choices. Physical action is used where it helps reveal structure: the learner moves through the route, handles the instruction, selects registers, works with datapackets, and completes local validation. Other forms of support remain in panels, prompts, and concise references because they are more useful there than they would be as further physical tasks. In practice, this allows the environment to remain active without becoming cumbersome.

Local guidance plays an important role in that balance. Panels, hints, and nearby prompts form part of the teaching strategy through which the route remains legible while the learner is still acquiring the structure of the lesson. The environment asks the learner to act where action clarifies the concept, but it does not ask the learner to carry every explanatory burden unaided at every moment.

4.5.3 Validation and Conceptual Discipline

The environment does not simply let the learner drift through the route and discover at the end that an early interpretation was wrong. Important decisions are checked near the point at which they are made so that an early misunderstanding does not propagate into later phases. Progress therefore depends on reasoning through the route coherently, not simply on reaching its final station.

4.6 Core Design Decisions and Trade-Offs

Several trade-offs define the final form of the prototype, the most important of which concerns pedagogical clarity and literal fidelity, a balance established earlier in the chapter. The remaining trade-offs concern breadth, interaction, and participant use.

Another trade-off concerns breadth and coherence. A wider system could have covered more instructions, more architectural topics, or a more detailed account of control and low-level circuitry, but that breadth would have weakened the central lesson loop and made it harder to defend the educational scope of the build.

There is also a trade-off between automation and learner action. If too much of the datapath remains automatic, the learner becomes a spectator. If too much is turned into interaction, the environment becomes cluttered and tiring. Much of the design work lay in deciding which decisions were worth preserving as learner action and which relationships were better carried by the environment itself. This balance also reflects a broader lesson from the literature: immersive space does not remove the need for restraint in information design [21, 23].

Finally, the design was shaped by the demands of participant use. This prototype does not have the freedom of an indefinitely expanding research platform. It had to become stable, navigable, and teachable within the available scope, which meant that features had to earn their place pedagogically and practically. Some ideas remained outside the final build not because they were uninteresting, but because they did not strengthen the educational core strongly enough to justify the complexity they would have introduced.

These trade-offs define the final design: a selective representational model shaped by pedagogical clarity, interaction burden, scope, and the practical demands of participant use.

4.7 Summary

This chapter defined the prototype as a supplementary learning environment built around a bounded set of learning outcomes for selected single-cycle MIPS datapath concepts. It established the decision to reorganise instruction execution into six learner-facing phases and showed how this shaped the routed environment, the progression across Learning mode, Practice mode, and Test mode, and the use of physical interaction, local guidance, and validation. The resulting design prioritises the reasoning structure needed by the learner while balancing architectural fidelity against clarity, interaction burden, and practical scope. Chapter 5 turns from these design decisions to their implementation in the prototype used for the participant study.

Chapter 5

Implementation

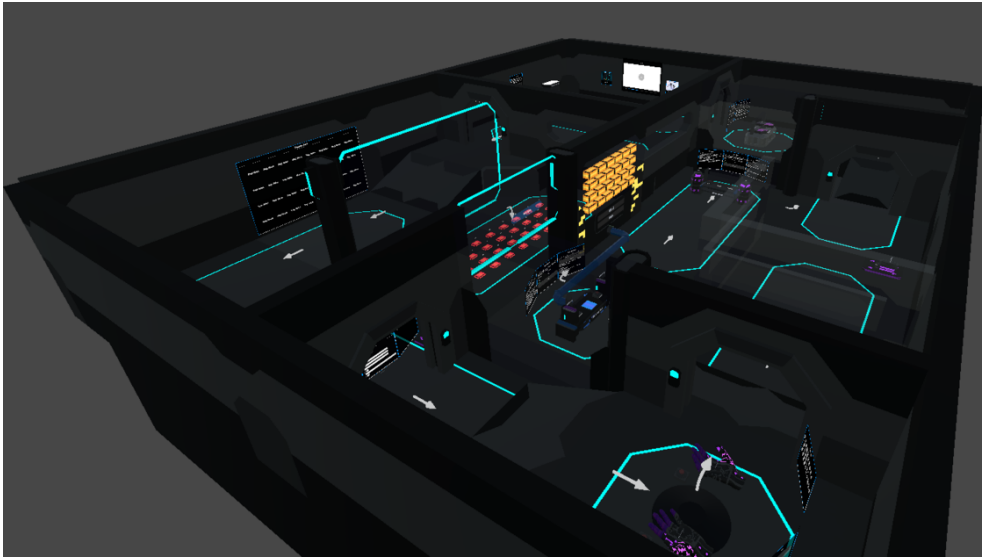


Figure 5.1: Overhead view of the complete VR lesson environment used in the participant study. The routed scene separates instruction execution into distinct learner-facing areas so that the learner can move through the lesson from instruction fetch toward later datapath consequences in a controlled sequence.

This chapter describes how the design was realised in the final prototype used for the participant study. The build does not reconstruct Figure 2.1 as a three-dimensional machine; it implements a spatial route through local teaching areas coordinated by a shared runtime that keeps the active instruction, lesson mode, progression state, validation rules, and architectural state aligned throughout the walkthrough.

The prototype, as shown in Figure 5.1, is examined here through the structures that make that lesson possible: the shared lesson framework that keeps the route coherent, the environment through which the session unfolds, the objects and interfaces that represent architectural state, the six phases through which instruction execution is carried, and the three-mode variation that formed the final study baseline.

5.1 Platform and Development Context

The prototype was developed in Unity v. 6000.4.11f1 and deployed as a Virtual Reality application. The build relies on Unity’s XR Interaction Toolkit v. 3.4.1 and the Unity VR Template as part of the baseline interaction and project setup foundations, alongside OpenXR support v. 1.16.1 and Meta OpenXR support v. 2.5.0 for the VR systems required by the staged lesson [24, 25].

The broader visual environment was assembled with the help of free Unity Asset Store packs, most notably the Free LowPoly SciFi Pack by Cuboom [26] and 3D Sprite Sci-Fi Props by Oiva Kahri [27]. These assets reduced the amount of custom modelling required and supported the creation of the environment, props, and interactable objects. The project source repository for the final build is available at <https://github.com/Norged-Out/CompArchVR>.

5.2 Overall System Architecture

At the broadest level, the prototype is organised around one shared lesson structure that keeps the active instruction, lesson mode, phase progression, validation state, and resulting architectural changes aligned across the whole route, implemented chiefly through `CpuLessonFlow`. This internal coherence allows the lesson to behave as one continuous walkthrough rather than as a set of disconnected stations. A learner-facing guide layer, anchored by `LessonGuideController`, ties that shared state to the visible panels, prompts, and support surfaces distributed across the route.

The shared runtime tracks the active instruction, current mode, next phase, validated decisions, and the state that must be passed onward. Phase-local systems activate only when the lesson flow reaches their part of the route and return control once the required work is complete. This allows the learner to follow one coherent account of execution while the implementation remains sequential at the lesson level, as presented in Figure 4.1.

This shared structure also supports all three lesson modes by reusing the same broad routes and central objects, while changing how much help is shown, how much must be reconstructed, and how strict later validation becomes. These mode-specific differences are governed centrally so that panel visibility, hint availability, and validation tolerance remain consistent throughout the route. The implementation directly realises the progression outlined in Chapter 4 and summarised in Figure 4.3: support is reduced across Learning mode, Practice mode, and Test mode, while the underlying lesson remains recognisably the same.

The prototype thus represents a bounded instructional system driven by au-

thored lesson definitions that store the identity, operand structure, and expected lesson behaviour of each supported instruction, rather than a full processor simulator. Guided lesson definitions, stricter encoded-instruction variants, and local reference content all feed the same broader walkthrough rather than branching into unrelated sub-systems.

5.3 Environment Structure and Lesson Routing

The entire lesson is staged inside one authored scene that functions as part of the teaching structure. As Chapter 4 argued, and as Figure 2.1 makes clear, the standard single-cycle datapath is compact, simultaneous, and densely connected. The final environment redistributes the major stages of instruction execution into a progressive route that preserves the broad order of the datapath while giving each local decision a clearer setting in which to be understood.

5.3.1 Scene Organisation

As Figure 4.2 shows, each major phase occupies its own authored zone within the route. The lesson begins in an introduction and fetch area that establishes the mode, provides the initial interface context, and stages the opening handoff of the instruction. Decode sits beside the register bank because that is the point at which encoded fields begin turning into specific operand choices and immediate handling. Execute follows as a distinct station, allowing ALU-related reasoning to be treated locally. Memory access sits beside the visible data memory structure, while write-back and Program Counter (PC) update occupy the later parts of the route so that final architectural consequence and control flow can be experienced as the closing stages of the lesson.

This organisation isolates each task without breaking the broader execution sequence. Placing Decode beside the persistent register bank makes register identity and operand selection spatially stable, while the Execute station separates incoming values, control-facing choices, and the resulting ALU output. Memory access has enough local space to distinguish the calculated address, stored word, and transferred value. Program Counter Update occupies a separate concluding area so that branch and jump consequences can be treated explicitly before the route ends. The learner therefore encounters one part of the execution process at a time while remaining within a single continuous walkthrough.

5.3.2 Guidance, Gates, and Local Interfaces

Two environmental systems help maintain that route during use. As shown in Figure 5.2, the first is a navigational guidance system that activates directional arrows

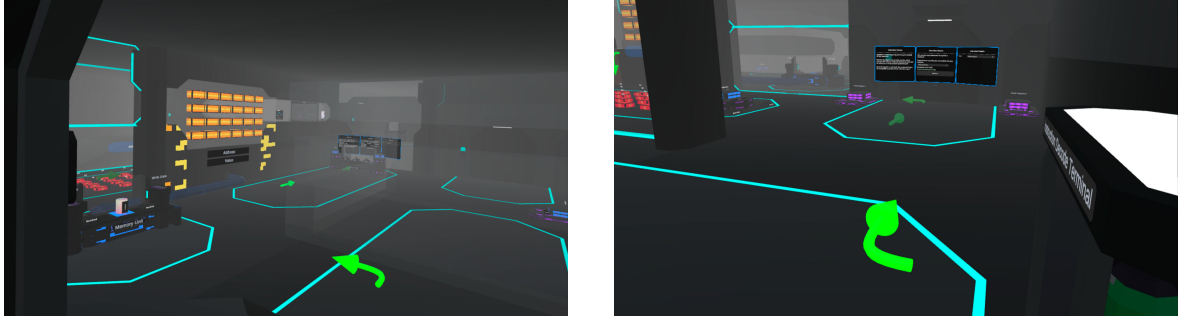


Figure 5.2: Examples of the directional guidance cues used to indicate the learner’s next destination during the lesson. These cues reinforce the intended phase order and reduce the need for the learner to infer the route through the environment independently.

pointing toward the next relevant destination and movement route. The second is a gate system that opens and closes parts of the route as the lesson progresses, giving controlled access to the areas of the route. Together they realise, in concrete scene form, the progression support argued for in Chapter 4.



Figure 5.3: Example of the three-panel interface used at major lesson stations. The panels separate task guidance, current interaction state, and reference information so that support can be presented locally without placing the entire datapath or lesson explanation in one display.

As shown in Figure 5.3, each major phase after fetch uses three nearby world-space panels so that local support can be split by function rather than collapsed into a single screen. The left panel provides phase-specific explanation and task requirements, the centre panel displays the active interaction state together with any validation demands or outcomes, and the right panel provides reference material or hints where

permitted by the current mode. Their contents are updated from the shared lesson state so that only information relevant to the active instruction, phase, and mode is shown, while still keeping enough nearby context for the learner to act, check, and continue without constantly reconstructing the wider lesson state.

5.4 Representation of Architectural State

One of the central implementation problems concerned the appearance of architectural state once the datapath had been translated into VR. In the standard diagram, some of the most important relationships are carried by buses, register-file reads, memory access, multiplexers, and control decisions that remain visually compact or fully implicit. The final build had to represent enough of that structure to support reasoning while remaining usable as a guided lesson rather than collapsing into manual busywork. The solution was a family of visible objects and interactions that externalise selected relationships without attempting to surface every internal detail literally.

5.4.1 Instruction Representation

The opening representation of the active instruction appears during fetch as a physical instruction module transferred between two terminals. In textbook single-cycle terms, this stands in for the point at which the current instruction becomes available from instruction memory under program-counter control, stored in the instruction register. Within the lesson, that availability is staged as a concrete handoff that marks the learner’s entry into the route.



Figure 5.4: The instruction module before lesson activation and after successful validation at the end of the implemented fetch phase. The state change gives the learner a visible indication that the selected instruction has been acquired and is ready to be carried forward into decode.

Figure 5.4 shows the instruction module first in its inactive state before a lesson begins and then after successful validation at the end of the fetch stage. The module

is best understood as a pedagogical stand-in for fetched-instruction availability, giving the route a definite beginning and making the transition from instruction selection to instruction interpretation easier to follow.

5.4.2 Register Representation

The register file is represented as a physical 32-register bank placed beside the decode region (Figure 5.5), accompanied by a larger state display that presents the same contents in a more readable overview. Each register remains a persistent architectural object with a stable identity and stored value, which directly supports one of the main learning outcomes established in Chapter 4: recognising which registers a supported instruction reads from or writes to and connecting those identities to later state change.

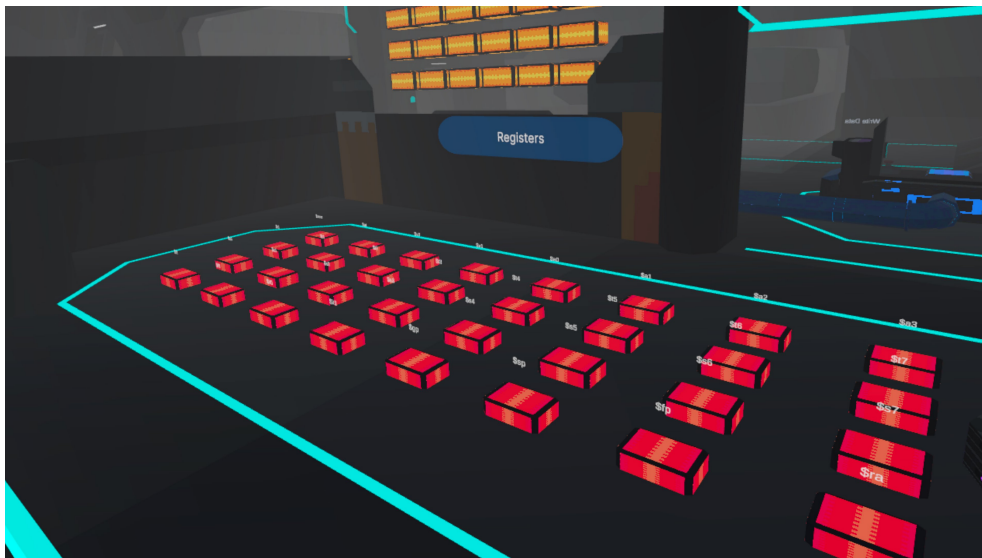


Figure 5.5: The physical register bank used during decode and later state verification. Persistent register identity is carried by visible scene objects rather than by labels alone.

In the standard datapath, register access is compact and mostly internal, and the significance of **rs**, **rt**, or **rd** is usually inferred from field labels and a small register-file box. Here, the bank gives those relationships a stronger presence, as the source and destination registers can be approached, selected, validated, and revisited. That visibility matters later as well, because write-back is easier to understand as an actual update to the architectural state already in play than as a merely local station event.

5.4.3 Datapackets and Value Flow

Once the lesson moves beyond register selection, the focus shifts to which value is now moving and what later phase depends on it. The implementation answers that

problem through datapackets. A datapacket is a portable learner-facing object that carries a value already produced or retrieved, whether that value comes from a register, an immediate, the ALU, memory, or a later control-flow calculation.

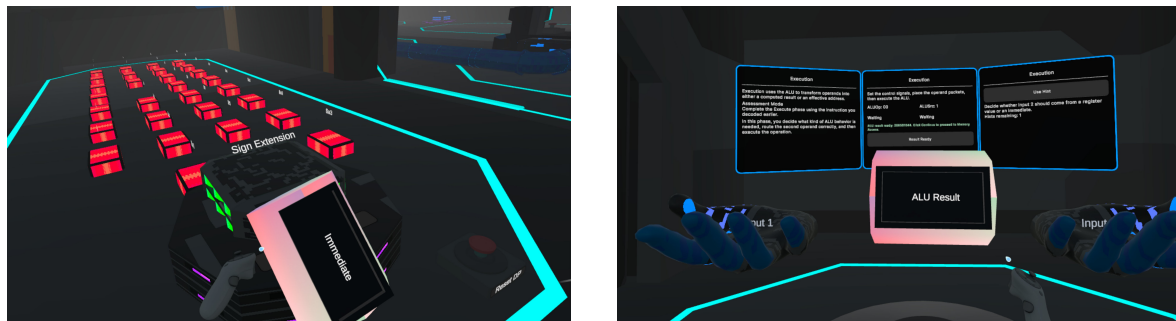


Figure 5.6: Datapackets produced after immediate extension and ALU execution. These portable objects externalise values that would otherwise move through internal datapath connections, allowing the learner to see which value was produced and carry that same value into the next relevant phase.

Figure 5.6 shows this translation clearly. Instead of asking the learner to imagine internal buses carrying `Read Data 1`, `Read Data 2`, an immediate, an ALU result, or a memory value, the build gives those states a portable physical identity. This helps abstract the hardware by preserving the reasoning structure that later stages depend on: a value was sourced or produced here, and that exact value must now be carried onward correctly.

This is also where sign extension becomes pedagogically visible. In an ordinary datapath explanation, the immediate may be shown entering an extender and then simply treated as ready for later use. In the implemented lesson, the immediate is externalised through its own transformed datapacket so that the learner can see that a raw immediate is not merely copied forward unchanged. The route therefore makes sign extension legible as part of operand preparation rather than leaving it as a silent internal conversion.

5.4.4 Scanner Representation

Scanners turn value recognition and local validation into explicit learner actions. A scanner, such as one shown in Figure 5.7, accepts either a register token or a datapacket, checks that it matches the expected role for the current instruction, and then either produces the next required state or unlocks the next relevant transition. This is one of the clearest ways in which the implementation redistributes otherwise hidden datapath relationships into visible lesson work.

In a standard account, reading from the register file, supplying the correct ALU

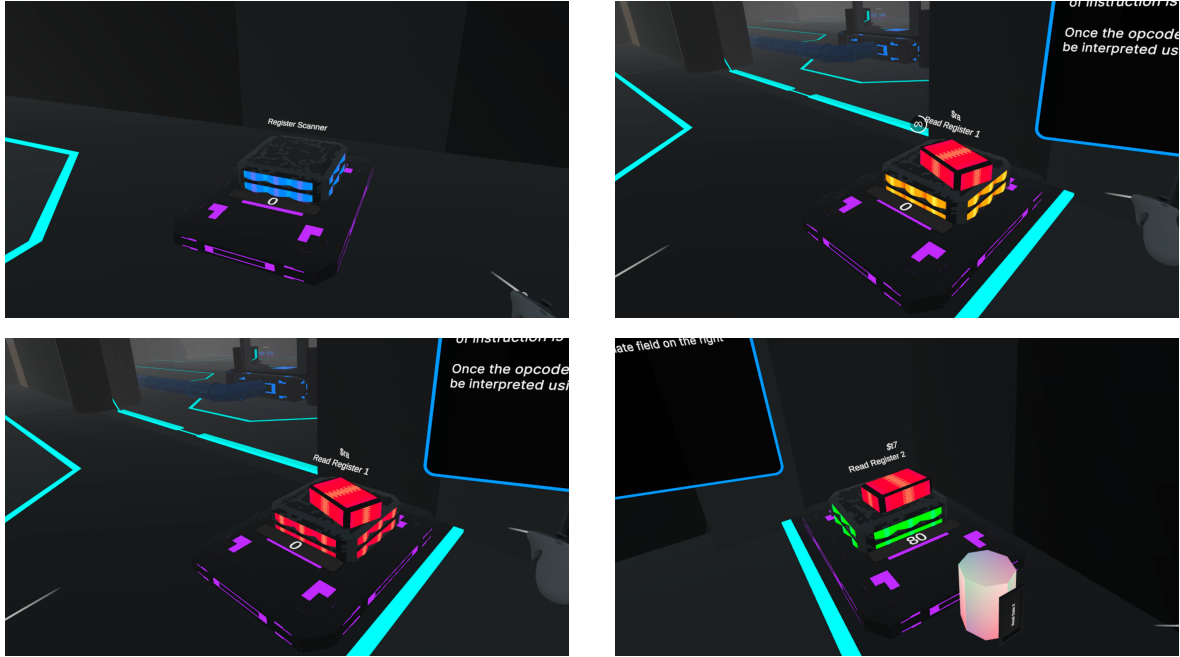


Figure 5.7: Scanner states during decode, including the idle scanner, an occupied state, failed validation, and successful validation followed by datapacket release. The sequence shows how scanners make operand selection and correctness checking into explicit learner actions before the lesson can progress.

operand, forwarding data for memory access, or passing the correct value into write-back are all internal connections. The build preserves the importance of those relationships, but asks the learner to complete them locally and correctly. The scanner therefore functions both as an interaction device and as an instructional mechanism: it checks the learner’s reasoning near the point at which the relevant architectural relationship matters.

5.4.5 Control Signals

Control is represented less as one hidden learner-facing control unit than as a distributed set of explicit choices that appear in the later phases of the route. In the textbook datapath shown in Figure 2.1, decode largely determines how execute, memory, write-back, and program-counter (PC) behaviour should proceed, often through centralised signal generation and multiplexer selection. The prototype does not automate this process, instead choosing to surface selected control-facing decisions where doing so helps the learner understand why later stages behave as they do.

If every control consequence remained internal, the later phases would risk becoming passive confirmations of a decision already made elsewhere. By distributing certain control-facing choices across execute, memory access, write-back, and program-

counter update, the implementation gives those later phases clearer instructional weight while still keeping the overall route within a bounded scope.

5.4.6 Memory Representation

Data memory is represented as a visible wall of authored memory words, coupled to a local address-and-value display and to transfer behaviour at the memory-access station. This gives memory a persistent place in the centre of the scene rather than reducing it to a hidden lookup or a panel-only display. The memory wall stands in for address-dependent access to stored data while giving the learner a visible distinction between address, stored word, and transferred result.

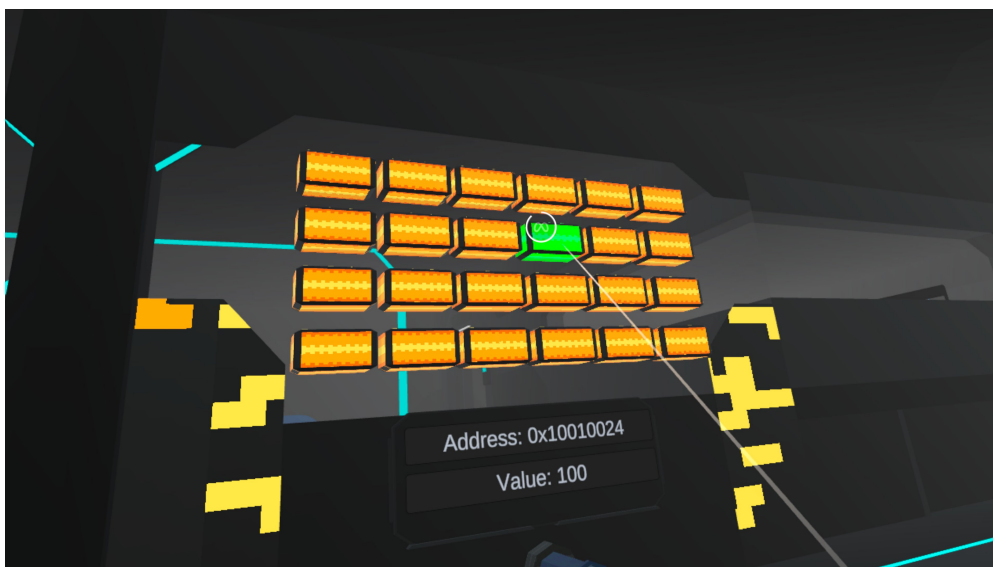


Figure 5.8: The data-memory wall used during memory access. Presenting addresses and stored words as a persistent scene structure helps the learner distinguish the calculated address, the selected memory location, and the data transferred by load or store behaviour.

Load and store instructions often cause difficulty because they require the learner to connect register values, offsets, addresses, stored contents, and later updates into one chain of reasoning. By making memory visible in place, as in Figure 5.8, the implementation gives that chain a stronger local structure than a compact internal datapath branch usually provides on a first pass.

5.5 Implementation of the Lesson Phases

The route is experienced as six implemented lesson phases that realise, in slower and more local form, the broad logic already outlined in Figure 4.1. Each phase gives one

part of instruction execution its own objects, interactions, and validation rules. What is followed here is the reasoning of instruction execution as it was staged for a guided supplementary lesson rather than the literal timing of the hardware itself.

5.5.1 Instruction Fetch

Fetch begins in the introduction area, where the active lesson mode and current instruction context are already known. The learner activates the fetch terminal, receives the instruction module, and carries it to the decode terminal to begin the rest of the route, as shown in Figure 5.9. This is a clear pedagogical staging of a process that the textbook datapath handles internally and with much greater simultaneity.

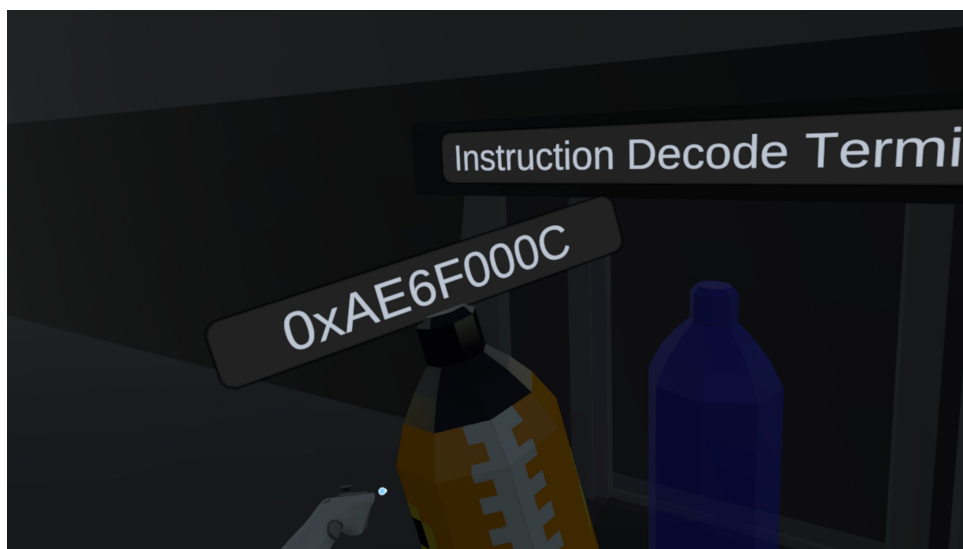


Figure 5.9: The instruction module being docked at the decode terminal after its transfer from the fetch area. This handoff marks the learner’s transition from acquiring the active instruction to interpreting its encoded fields and preparing operands.

The implemented fetch phase gives the route a definite beginning. The learner undergoes an explicit transition from instruction availability to instruction interpretation, and the remainder of the lesson is anchored to that opening handoff. The phase is deliberately simplified to provide a clear entry point to the lesson, with the module acting as a physical but non-literal representation of fetched-instruction availability.

5.5.2 Decode and Operand Preparation

Decode is the densest part of the implementation because it is where the instruction stops being an inert object and becomes the usable lesson context for everything that follows. In Learning mode, the phase introduces the instruction structure step by step,

surfaces the relevant fields, and prepares the learner for source-register selection and any immediate-related work.

Practice and Test change the point of entry into this phase, as illustrated for Practice mode in Figure 5.10, with the learner seeing an encoded instruction that must be reconstructed with less or no support before later stages become available. This is where mode variation is strongest, because decode is the place at which the instruction first becomes actionable lesson state rather than merely a selected item.

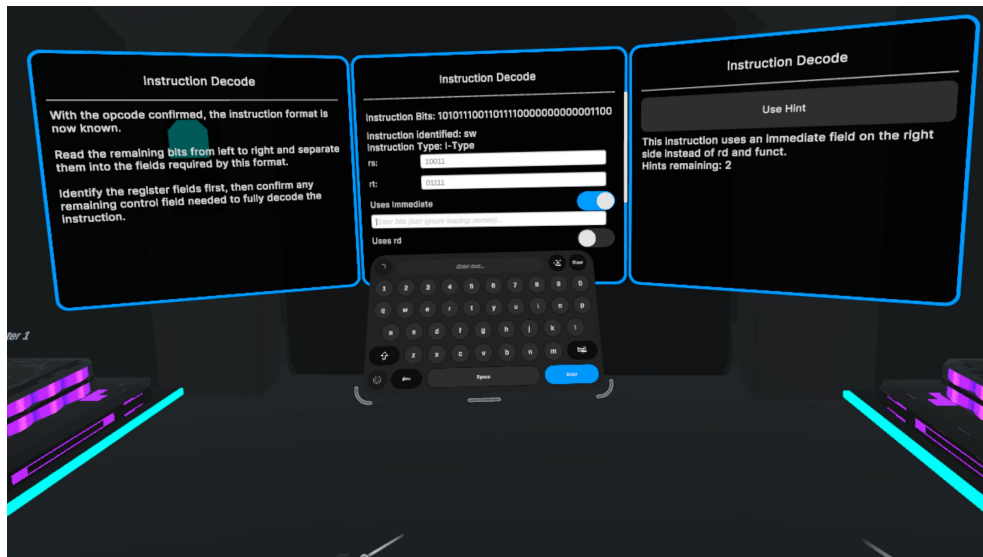


Figure 5.10: Decode in Practice mode, where the instruction is presented in encoded form and the learner must reconstruct the relevant field structure with less support than in Learning mode. The figure illustrates how the same decode task is retained while explanatory scaffolding is reduced.

Operand preparation then carries that state forward, as the learner retrieves the required source registers from the bank, places them on the correct scanners, and receives the resulting datapackets used downstream. Where an immediate is needed, the lesson also prepares that value in a form suitable for later use through the extender-linked transformation already shown in Figure 5.6, allowing sign-extension-related behaviour to become visible rather than remaining a quiet internal transformation.

5.5.3 Execute

Execute is realised as a dedicated ALU station in which operand interpretation, operation selection, and result production become local tasks. As shown in Figure 5.11, the learner must supply the appropriate values, set the relevant control-facing options (ALUSrc and ALUOp), and complete the validation required for the active instruction before the result datapacket (ALUResult or ZeroResult) is produced. This gives ALU

reasoning enough local presence without overloading the user with information pertaining to logic gates or surfacing mux-like decisions.



Figure 5.11: Execute-phase validation in Learning mode, showing operand handling and the learner-facing ALU decisions required for the active instruction. The interaction makes operand choice, `ALUSrc`/`ALUDst`-related reasoning, and result production visible before the lesson continues.

In the standard datapath, ALU behaviour is often recognised in principle before it is understood in use. By slowing the process and making later consequences depend on validated operand and control choices, the station exposes ALU reasoning that remains compact in the conventional datapath.

5.5.4 Memory Access

Memory access handles the point at which the active instruction either interacts with data memory or passes onward without needing it, as shown in Figure 5.12. For load instructions, the learner must establish the correct address path before the memory wall reveals the targeted word and transfers the resulting data onward. For store instructions, the learner must supply both the correct address and the correct data before the update can occur. Instructions that do not use data memory are allowed to pass through without artificial interaction, which keeps the phase aligned with the actual architectural differences between instruction classes.

The phase also surfaces the relevant control-signal decisions through `MemRead` and `MemWrite`, making the distinction between read and write behaviour part of the learner-facing interaction. Address calculation, stored value, loaded value, and later result use all appear here as connected but locally distinguishable ideas. Once a store



Figure 5.12: The memory-access station during reduced-support use. Address validation, memory-specific control choices, and the resulting load or store consequence are handled within one local interaction space so that the direction and effect of the memory operation remain visible.

updates memory, that change remains part of the lesson state in the data memory. Once a load retrieves a word, the resulting datapacket can be understood as coming from a visible memory location rather than from an abstract internal branch of the diagram.

5.5.5 Write-Back

Write-back became a full physical phase in the final build where the learner must identify whether register write-back should occur, determine the correct destination register, supply the appropriate result source, and validate the resulting update to the register bank. This gives the route a clear culminating moment at which earlier decisions resolve into a visible architectural consequence.

Figure 5.13 shows why this phase carries more weight than a compact textbook write-back event usually does on first encounter. It brings together the main interaction patterns established earlier in the route. The learner must confirm `RegWrite`, use `RegDst` to determine the correct destination register, and use `MemToReg` to determine the correct result source.

The implemented station gathers together work completed earlier in the route and makes the final update readable through both local interaction and the larger register-state display. Instructions that do not write to the register file simply bypass this stage, which preserves the selective structure present in the standard datapath.



Figure 5.13: Write-back interaction in Learning mode, where the learner identifies whether register write-back should occur, selects the destination and result source, and validates the register update. The phase makes the final architectural consequence of earlier datapath decisions visible at the register bank.

5.5.6 Program Counter Update

The final phase handles next-PC reasoning, branch consequence, and the limited jump behaviour supported in the study build. In a strict single-cycle account, these relationships are resolved in parallel within the same overall cycle, alongside other internal work. The implementation reorganises them into a dedicated concluding station so that control-flow consequence has a clear instructional place in the route.

Figure 5.14 illustrates the logic of that decision. The aim is control-flow legibility over literal timing fidelity. Branch and jump behaviour can be difficult to absorb when control-flow consequences remain implicit or compressed into the same representation as the rest of the datapath. By giving next-PC reasoning its own station, the lesson allows the route to close with a clear account of where execution would go next and why.

Program Counter Update is the clearest example of pedagogical serialisation in the build. Next-PC reasoning is separated from the overall single cycle so that the learner can inspect the control-flow consequence and relevant target calculation as a distinct teaching step.



Figure 5.14: Program-counter update in Learning mode, where next-PC and branch-related consequences are treated as an explicit final lesson decision. Separating this reasoning from the earlier phases gives the learner a visible account of where execution proceeds next and why.

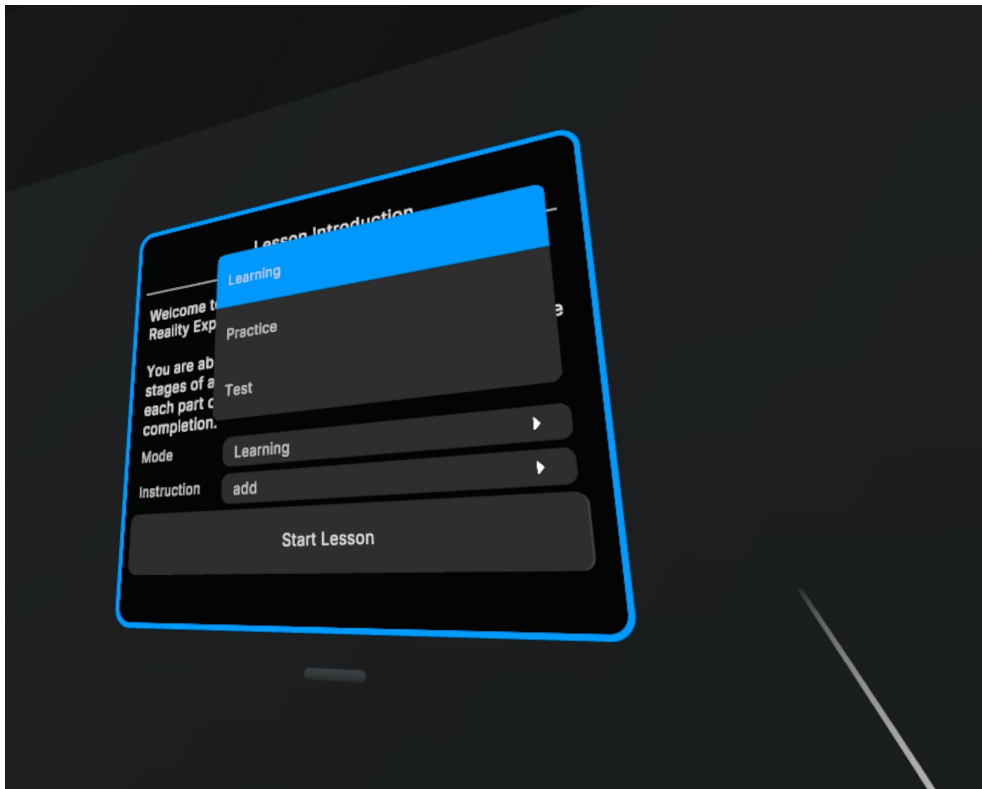


Figure 5.15: The lesson-introduction interface used to select Learning, Practice, or Test mode before instruction execution begins. The selected mode determines how much guidance, tolerance, and reference support is available while preserving the same overall lesson route.

5.6 Implementation of the Three Lesson Modes

The three lesson modes share one route and one broad lesson architecture while changing the amount of support, tolerance, and independent reconstruction expected from the learner. Figure 5.15 shows the shared lesson-entry interface through which the modes are selected. In implementation terms, this realises the pedagogical progression already described in Chapter 4 and shown in Figure 4.3.

5.6.1 Learning Mode

Learning mode provides the fullest level of support. Guided instruction assets, phase explanations, reference material, and control-signal guidance remain available throughout the route, while validation is comparatively tolerant. Its purpose is to establish the interaction conventions and reasoning sequence before support is reduced in later modes.

In implementation terms, Learning mode also provides the baseline lesson configuration from which Practice and Test derive their shared assets, flow steps, and explanatory content.

5.6.2 Practice Mode

Practice mode reuses the same downstream route but changes the entry conditions and the strictness of later interactions. The instruction is shown in hexadecimal encoded form during fetch and in 32-bit binary form in decode, and the learner must complete a typed bitfield decode before the rest of the phase sequence becomes available.

The largest change occurs during Decode, where the learner must reconstruct the encoded instruction before continuing. Later phases use tighter attempt budgets, limited hints that respond to the learner’s progress, and persistent failure states. These differences are implemented through shared Practice-mode policy systems rather than through duplicated phase controllers.

5.6.3 Test Mode

Test mode is the strictest version of the lesson. It reuses the assessment foundation built for Practice mode while removing most of the remaining support. Instruction selection becomes randomised from the supported practice pool, hint surfaces are entirely disabled, lesson-facing support panels are entirely hidden, and validation tolerance is reduced to one mistake per relevant category in each phase, with each failure forcing a full lesson failure.

This mode did not require a new architectural path, as it is still the same datapath lesson route. The learner completes the same route under conditions intended to show whether the earlier supported reasoning can be reconstructed with minimal assistance.

5.7 Implemented Content Baseline

The participant-study build supported a deliberately limited set of arithmetic, immediate, memory, branch, and jump cases. This baseline was wide enough to show that different instruction classes activate different parts of the datapath and therefore demand different forms of reasoning, while still remaining narrow enough for the lesson to stay stable, testable, and interpretable within the scope of the study.

5.7.1 Supported Instruction Set

Tables 5.1 and 5.2 summarise the instruction sets authored for Learning mode and Practice mode respectively.

Instruction	Assembly
add	add t0, t1, t2
addi	addi v1, t3, 4
lw	lw at, 12(s2)
sw	sw t6, 12(s4)
sub	sub t5, t3, t6
and	and t8, t4, t7
or	or t9, a0, a1
slt	slt v0, a2, a3
beq	beq k0, k1, 8
bne	bne a0, a1, 8
j	j target

Table 5.1: Instruction set authored for Learning mode, showing the supported MIPS operations and their associated representations in the prototype. The set provides guided examples across multiple instruction classes so that the learner encounters different datapath behaviours within the same lesson structure.

Test mode reuses the same ten-instruction Practice-mode pool, but under stricter conditions: instruction selection is randomised at lesson start, lesson and hint panels are hidden, each phase allows only one validation failure and one scanner failure, and no hints are available. The point of this instruction set was not to suggest broad coverage of MIPS as a whole, but to provide enough variation for differences in operand

Instruction	Encoded Display	Runtime Assembly
add	0x018D5820	add t3, t4, t5
addi	0x201F0010	addi ra, zero, 16
lw	0x8EA2000C	lw v0, 12(s5)
sw	0xAE6F000C	sw t7, 12(s3)
sub	0x03C15022	sub t2, fp, at
and	0x03BE1824	and v1, sp, fp
or	0x0026C025	or t8, at, a2
slt	0x0008082A	slt at, zero, t0
beq	0x105B0004	beq v0, k1, 4
bne	0x14C70004	bne a2, a3, 4

Table 5.2: Instruction pool available in Practice mode. The pool retains multiple arithmetic, immediate, memory, and control-flow cases so that reduced-support activity requires learners to apply the established lesson structure across different datapath behaviours rather than repeating a single example.

handling, immediate use, memory behaviour, write-back, and control flow to become legible within the lesson.

5.7.2 Authored Register and Memory Baseline

Tables 5.3 and 5.4 show the fixed register and data-memory states used by the study build.

The authored register and memory baseline matters because the lesson depends on interpretable state rather than arbitrary values. The learner needs to be able to see why a particular operation produced a particular result, where a load retrieved its value from, why a store updated the word it did, and how those changes relate back to the route they have just completed. Persistent authored state also helps the implementation support coherence across repeated lessons instead of reducing each phase to a self-contained demonstration.

5.8 Technical Constraints and Deliberate Simplifications

The final implementation reflects several boundaries that were both practical and intentional. The most obvious is scope. The prototype is centred on selected single-cycle MIPS instruction execution and does not attempt to cover the whole of assembly programming, multicycle behaviour, pipelining, hazards, or gate-level digital logic. That

Register	Value	Register	Value
\$zero	0	\$t8	90
\$at	1	\$t9	100
\$v0	0	\$k0	0
\$v1	0	\$k1	0
\$a0	4	\$gp	0x10010000
\$a1	8	\$sp	8192
\$a2	12	\$fp	8192
\$a3	16	\$ra	0
\$t0	10	\$s0	0x10010004
\$t1	20	\$s1	0x10010010
\$t2	30	\$s2	0x1001001C
\$t3	40	\$s3	0x10010028
\$t4	50	\$s4	0x10010034
\$t5	60	\$s5	0x10010040
\$t6	70	\$s6	0x1001004C
\$t7	80	\$s7	0x10010058

Table 5.3: Authored logical register values used as the baseline state of the final prototype. The fixed values provide a stable and interpretable starting state so that operand selection, ALU results, and later register updates can be traced consistently during lesson execution.

Word	Address	Value	Word	Address	Value
Word 1	0x10010000	10	Word 13	0x10010030	130
Word 2	0x10010004	20	Word 14	0x10010034	140
Word 3	0x10010008	30	Word 15	0x10010038	150
Word 4	0x1001000C	40	Word 16	0x1001003C	160
Word 5	0x10010010	50	Word 17	0x10010040	170
Word 6	0x10010014	60	Word 18	0x10010044	180
Word 7	0x10010018	70	Word 19	0x10010048	190
Word 8	0x1001001C	80	Word 20	0x1001004C	200
Word 9	0x10010020	90	Word 21	0x10010050	210
Word 10	0x10010024	100	Word 22	0x10010054	220
Word 11	0x10010028	110	Word 23	0x10010058	230
Word 12	0x1001002C	120	Word 24	0x1001005C	240

Table 5.4: Authored data-memory baseline used by the final prototype, with word addresses shown in hexadecimal and their corresponding stored values. The fixed memory state gives load and store instructions predictable locations and values that can be traced visibly during the lesson.

narrowing was necessary for the lesson to stay coherent enough to support the educational claim being made.

Another boundary concerns fidelity to actual hardware behaviour. The build remains grounded in the logic of the textbook single-cycle datapath, but does not preserve its hidden, parallel, and largely internal nature in literal form. Instructions are physically moved, decode is stretched into learner work, values are carried as datapackets, and some control-facing decisions are distributed across later stations. These simplifications were adopted because the project is framed as a pedagogical VR abstraction rather than as a hardware-faithful simulator.

There were also more ordinary implementation constraints. Scene-authored systems are easier to tune locally, but they require careful serialised wiring and make many controller relationships sensitive to scene organisation. Decode in particular grew into the busiest area of the project because it sits at the meeting point of instruction identity, field interpretation, register preparation, immediate handling, and mode-specific difficulty. The prototype addressed that complexity by splitting support logic into helpers, while still keeping the whole lesson on one shared runtime.

5.9 Summary

The prototype was implemented as a scene-authored VR lesson system whose central concern is instructional coherence. One shared runtime coordinates instruction selection, lesson progression, local validation, and architectural state, while the environment itself presents the learner with a routed sequence of Fetch, Decode, Execute, Memory, Write-Back, and Program Counter Update stations.

The final study build includes the three-mode lesson structure, physical instruction handoff, register interaction, datapackets, immediate extension, ALU interaction, memory access, write-back, program-counter update, and the authored instruction, register, and memory baselines that support those systems. Together, these systems form the implementation taken forward into the participant study.

Chapter 6

Results

This chapter reports the findings from the participant study described in Chapter 3. The results are organised around five strands of evidence: participant background, the post-session questionnaire, post-session knowledge-check performance, the repeated baseline/post-session comparison, and qualitative material drawn from the open-ended questionnaire responses and researcher session notes. The full participant questionnaire is reproduced in Appendix A. Interpretation of these findings in relation to the research question and the literature reviewed in Chapter 2 is reserved for Chapter 7.

6.1 Participant Background and Study Context

The final study sample comprised 12 participants, all of whom confirmed that they were aged 18 or over. Table 6.1 summarises the background information collected before the VR session, including prior exposure to computer architecture or computer organisation, previous VR use, self-reported confidence and difficulty, and prior familiarity with selected topic areas.

Nine participants (75.0%) reported previous study of computer architecture or computer organisation, while 11 (91.7%) had previously used a VR headset. Previous VR experience was rated as Good by five participants and High by four, with one participant each reporting Moderate, Limited, and no prior experience.

Pre-session confidence was distributed across the full five-point scale: four participants selected rating 1, one selected rating 2, two selected rating 3, three selected rating 4, and two selected rating 5. Perceived difficulty was concentrated at the more difficult end of its scale, with one participant selecting Extremely difficult, six selecting Somewhat difficult, and five selecting Neutral; no participant selected either of the two easy categories.

Topic familiarity was also recorded before the session as part of the participant background profile. Stack was the most frequently selected topic, reported by nine

Category	Response	<i>n</i>	%
<i>Prior exposure</i>			
Computer architecture / organisation study	Yes	9	75.0
Previous VR headset use	Yes	11	91.7
<i>VR experience level</i>			
VR experience	None	1	8.3
VR experience	Limited	1	8.3
VR experience	Moderate	1	8.3
VR experience	Good	5	41.7
VR experience	High	4	33.3
<i>Prior computer-architecture confidence</i>			
Confidence rating	1	4	33.3
Confidence rating	2	1	8.3
Confidence rating	3	2	16.7
Confidence rating	4	3	25.0
Confidence rating	5	2	16.7
<i>Perceived prior difficulty</i>			
Difficulty rating	1	1	8.3
Difficulty rating	2	6	50.0
Difficulty rating	3	5	41.7
Difficulty rating	4	0	0.0
Difficulty rating	5	0	0.0
<i>Previously studied topics</i>			
Topics selected	Instruction Flow	3	25.0
Topics selected	Pipelining	5	41.7
Topics selected	Stack	9	75.0
Topics selected	Memory Hierarchy	8	66.7
Topics selected	Assembly	3	25.0
Topics selected	None of the above	2	16.7

Table 6.1: Background characteristics of the 12-participant study sample, including prior architecture exposure, VR experience, confidence, perceived difficulty, and previously studied topics. These measures describe the varied starting context from which participants entered the exploratory evaluation.

participants (75.0%), followed by Memory Hierarchy by eight (66.7%) and Pipelining by five (41.7%). Instruction Flow and Assembly were each selected by three participants (25.0%), while two participants (16.7%) selected None of the above. As described in Chapter 3, these background responses formed part of the starting context used when delivering the tailored refresher before the VR session.

6.2 System-Experience Results

The post-session questionnaire included six positively framed usability items, a separate discomfort item, five engagement items, and five items addressing the perceived learning experience. Table 6.2 reports group-level descriptive summaries, while Figures 6.1–6.4 retain the item-level response distributions. The full wording of each item is reproduced in Appendix A. Standard deviation (SD) is reported alongside the mean where applicable.

Grouping	Measure	<i>N</i>	Mean (SD)	Median	Range	Reliability
Usability	Six-item usability composite	12	3.99 (0.61)	4.17	2.50–4.67	$\alpha = 0.820$
Discomfort	Single discomfort item	12	3.92 (1.31)	4.00	1–5	Not applicable
Engagement	Five-item descriptive summary	12	4.23 (0.37)	4.20	3.60–5.00	$\alpha = 0.160$
Learning experience	Five-item learning-experience composite	12	4.38 (0.59)	4.60	3.20–5.00	$\alpha = 0.836$

Table 6.2: Descriptive statistics for the post-session usability, discomfort, engagement, and perceived learning-experience measures, together with internal-consistency estimates where applicable. The table provides group-level descriptive statistics while the accompanying figures retain the individual item distributions.

The usability and learning-experience groupings returned Cronbach’s α values of 0.820 and 0.836 respectively. The engagement items returned $\alpha = 0.160$, and their mean is retained here only as a descriptive summary alongside the individual item distributions.

6.2.1 Usability

The six-item usability composite had a mean of 3.99 ($SD = 0.61$), a median of 4.17, and a range of 2.50 to 4.67. Figure 6.1 reports the response distribution across the six statements. Eleven of the 12 participants (91.7%) selected Agree or Strongly Agree for navigation without difficulty, intuitive controls, and the statement that the visual presentation made the concepts easier to follow. Nine participants (75.0%) selected Agree or Strongly Agree for ease of learning and confidence in using the system again. The focus item had the lowest level of agreement within the usability group, with six participants (50.0%) selecting Agree or Strongly Agree, three selecting Neutral, and three selecting Disagree.

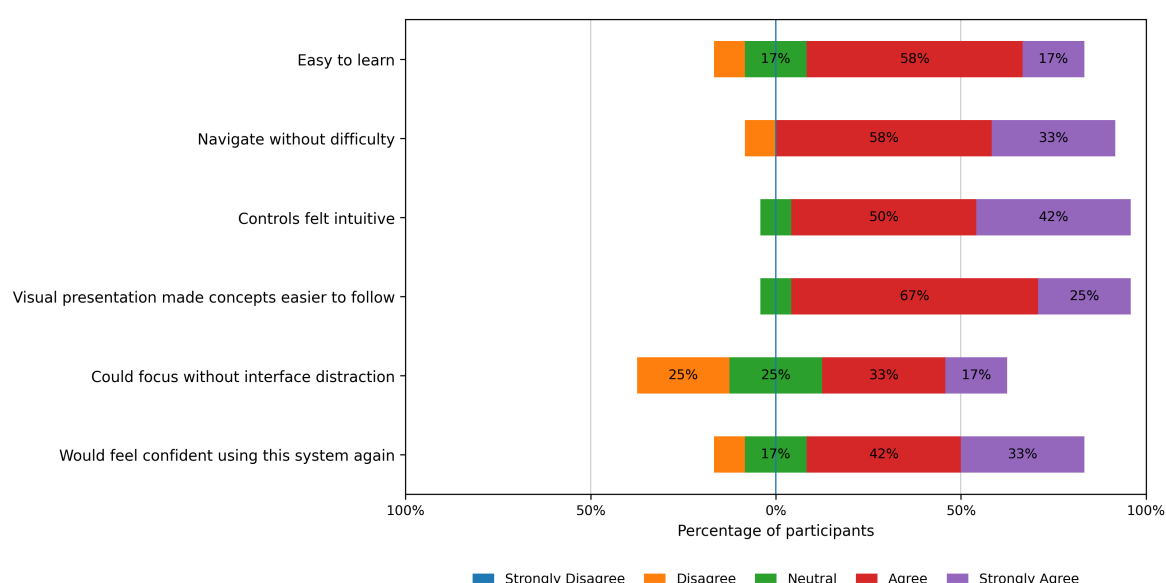


Figure 6.1: Distribution of participant responses across the six positively framed usability items following the VR session. The item-level view allows the individual usability responses to be compared rather than relying only on the overall usability composite.

6.2.2 Discomfort

The single discomfort item had a mean response of 3.92 ($SD = 1.31$), a median of 4.00, and a range of 1 to 5. As shown in Figure 6.2, nine participants (75.0%) selected Agree or Strongly Agree with the statement that they experienced discomfort, motion sickness, or visual fatigue while using the system. One participant selected Neutral, one selected Disagree, and one selected Strongly Disagree.

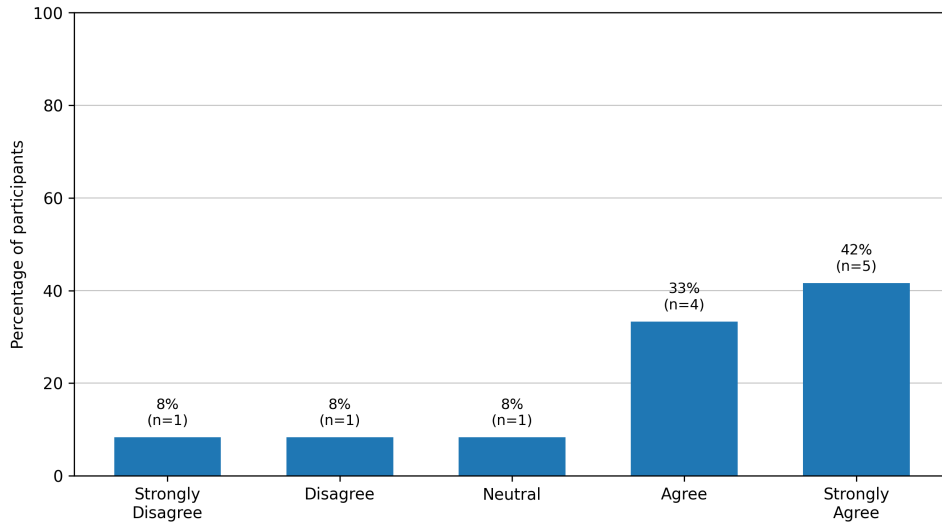


Figure 6.2: Distribution of participant responses to the post-session item concerning discomfort, motion sickness, or visual fatigue during VR use. The figure reports this physical-experience measure separately from the positively framed usability items.

6.2.3 Engagement

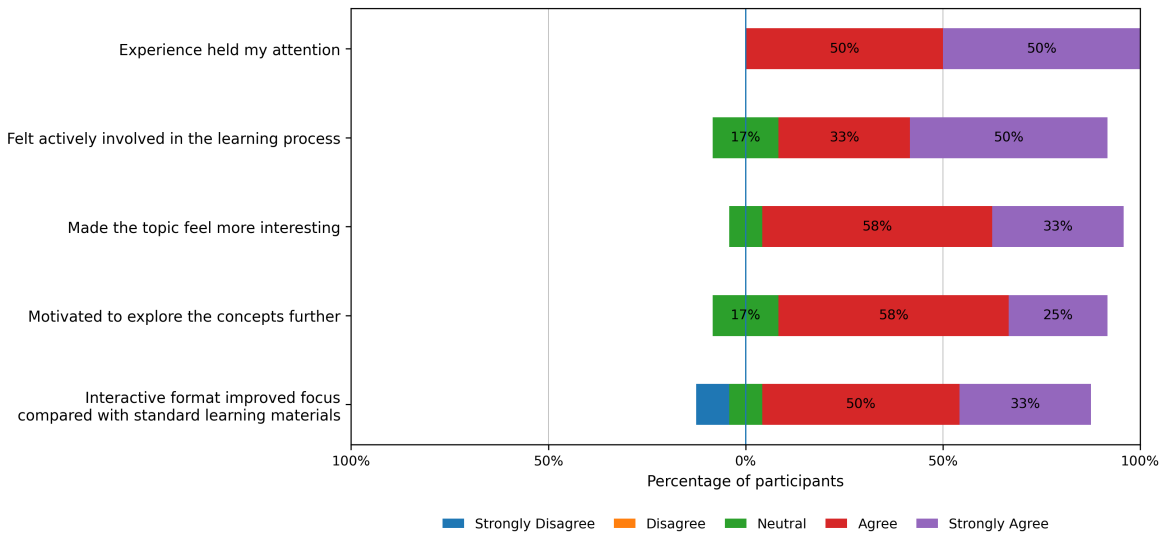


Figure 6.3: Distribution of participant responses across the five engagement items following the VR session. The individual item distributions are shown separately because the five engagement items had weak internal consistency and are therefore interpreted at item level, with the mean retained only for descriptive context.

For descriptive context only, the mean across the five engagement items was 4.23 ($SD = 0.37$), with a median of 4.20 and a range of 3.60 to 5.00. Figure 6.3 reports the individual response distributions. All 12 participants (100.0%) selected Agree or

Strongly Agree for the statement that the experience held their attention. Eleven participants (91.7%) selected Agree or Strongly Agree that the system made the topic feel more interesting. Active involvement in the learning process, motivation to explore the concepts further, and improved focus compared with standard learning materials each received Agree or Strongly Agree responses from 10 participants (83.3%). The final focus-comparison item also included one Strongly Disagree response.

6.2.4 Perceived Learning Experience

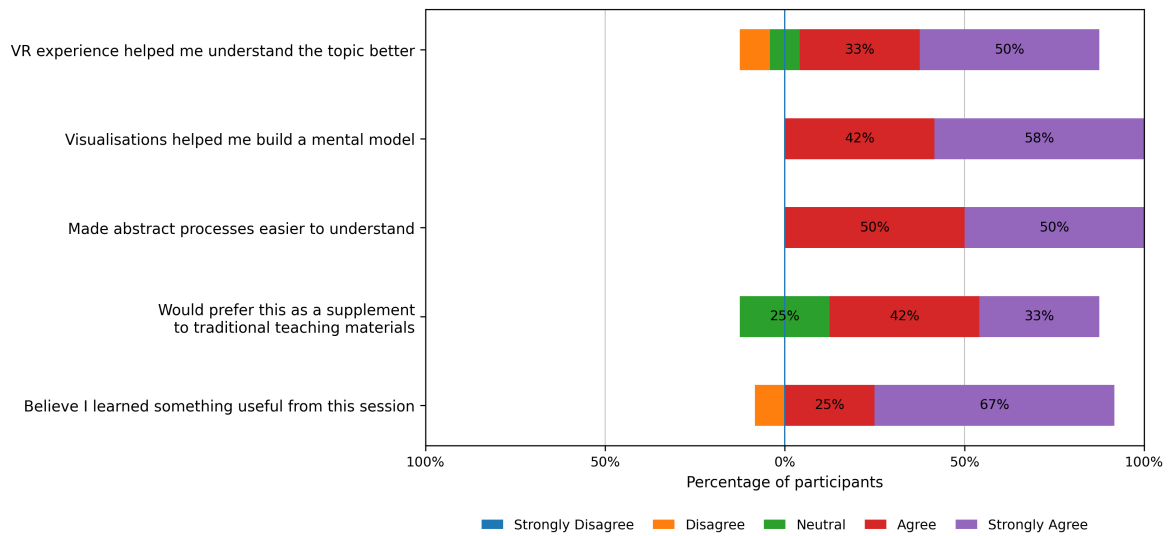


Figure 6.4: Distribution of participant responses across the five perceived learning-experience items following the VR session. The figure provides the item-level pattern for reported mental-model support, understanding of abstract processes, perceived usefulness, understanding, and supplementary-tool preference.

The five perceived learning-experience items produced a composite mean of 4.38 ($SD = 0.59$), a median of 4.60, and a range of 3.20 to 5.00. Figure 6.4 shows the item-level response distribution. All 12 participants (100.0%) selected Agree or Strongly Agree for the statements that the visualisations helped them build a mental model of what was happening and that the system made abstract processes easier to understand. Eleven participants (91.7%) selected Agree or Strongly Agree that they learned something useful from the session, while 10 (83.3%) did so for the statement that the VR experience helped them understand the topic better. Nine participants (75.0%) selected Agree or Strongly Agree that they would prefer the system as a supplement to traditional teaching materials.

6.3 Knowledge-Check Results

As described in Chapter 3, the post-session knowledge check provided a descriptive measure of participant performance on selected concepts after the overall learning session. Questions 11–25 covered program-counter behaviour, instruction fetch, the instruction register, stage order, the execute stage, branch-related reasoning, and the behaviour of supported MIPS instructions including `add`, `addi`, `lw`, `sw`, `beq`, and `j`. The exact wording and response options are reproduced in Appendix A.

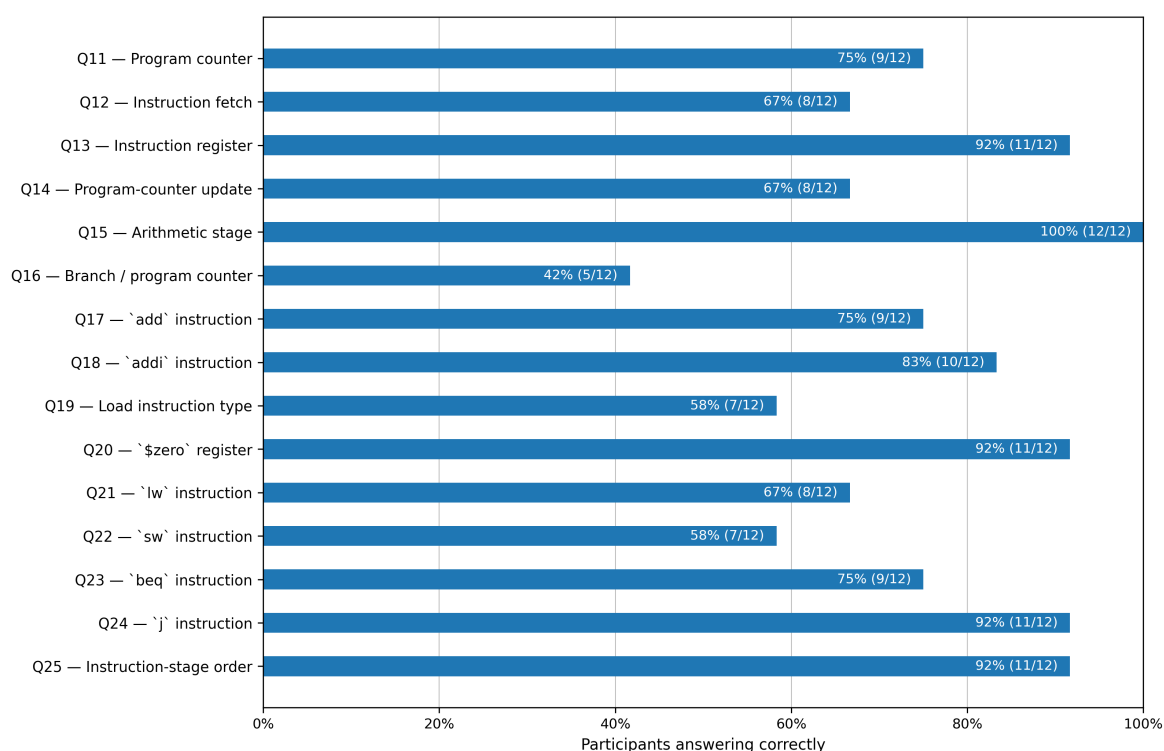


Figure 6.5: Post-session knowledge-check performance across Questions 11–25, showing the percentage and number of participants answering each item correctly. The item-level presentation makes it possible to identify which targeted concepts were answered more or less consistently across the sample rather than relying only on the overall knowledge score.

Table 6.3 reports the overall descriptive summary of post-session knowledge-check performance and confidence. Across the 15-item post-session knowledge check, the mean total score was 11.33 out of 15 ($SD = 2.02$), with a median of 10.50 and a range of 9 to 15. Expressed as a percentage, the mean was 75.56% ($SD = 13.43$), the median was 70.00%, and the observed range was 60.00% to 100.00%.

Question 26 asked participants to rate their confidence in the preceding knowledge-check answers. The mean confidence rating was 3.08 ($SD = 1.00$), with a median of 3.00 and a range of 1 to 4. One participant selected rating 1, two selected rating 2,

four selected rating 3, and five selected rating 4; no participant selected rating 5. Figure 6.5 reports the item-level percentage-correct results. Q15 was answered correctly by all 12 participants (100.0%). Q13, Q20, Q24, and Q25 each received 11 correct responses (91.7%), while Q18 received 10 (83.3%). Q11, Q17, and Q23 each received nine correct responses (75.0%); Q12, Q14, and Q21 each received eight (66.7%); Q19 and Q22 each received seven (58.3%); and Q16 had the lowest correct-response count, with five participants answering correctly (41.7%).

The incorrect-response distributions were also inspected for the items with repeated distractor choices. On Q16, four participants selected Instruction Register rather than Program Counter. On Q22, four participants selected the response describing a value being loaded from memory into `$t0`, while seven selected the correct store-word response. For Q12, all four incorrect responses selected Decode rather than Fetch. On Q17, the three incorrect responses selected the alternative that reversed the source and destination registers.

Measure	<i>N</i>	Mean (SD)	Median	Range
Total score out of 15	12	11.33 (2.02)	10.50	9–15
Percent correct	12	75.56 (13.43)	70.00	60.00–100.00
Post-session confidence rating	12	3.08 (1.00)	3.00	1–4

Table 6.3: Summary of post-session knowledge-check performance and confidence, reporting the total score, percentage-correct score, and self-reported confidence rating across the 12 participants. These measures provide an overall descriptive view of performance after the complete learning session.

6.4 Baseline and Post-Session Comparison

The baseline probe was administered before both the tailored refresher and the VR session, following the procedure set out in Chapter 3. It repeated five items from the later knowledge check: Q11, Q12, Q13, Q14, and Q25. The repeated set provided a limited comparison on the same foundational concepts at two points in the study procedure. Since the tailored refresher occurred after the baseline probe and before the VR session, the comparison reported in this section describes change across the overall learning session rather than change attributable to the VR environment alone.

Some participants left individual baseline items unanswered, and missing responses were retained as missing rather than scored as incorrect. The resulting matched sample sizes were 11 for Q11, 11 for Q12, 10 for Q13, nine for Q14, and 11 for Q25. Eight participants answered all five baseline items and therefore contributed to the

strict complete-case comparison. Table 6.4 reports both that complete-case analysis and the participant-specific available-item comparison used to retain the remaining baseline information.

Analysis block	Measure	<i>N</i>	Mean (SD)	Median	Range
<i>Strict five-item complete-case comparison</i>					
Complete cases	Baseline total out of 5	8	3.38 (1.19)	3.00	2–5
Complete cases	Post-session total out of 5	8	4.25 (0.89)	4.50	3–5
Complete cases	Difference out of 5	8	0.88 (0.83)	1.00	0–2
<i>Participant-specific available-item comparison</i>					
Available matched items	Number of matched items	12	4.33 (1.23)	5.00	1–5
Available matched items	Baseline matched percent	12	60.97 (27.56)	60.00	0.00–100.00
Available matched items	Post-session matched percent	12	74.72 (28.11)	77.50	0.00–100.00
Available matched items	Percentage-point difference	12	13.75 (15.83)	10.00	0.00–40.00

Table 6.4: Descriptive summary of the repeated baseline/post-session comparison using both the strict five-item complete-case sample and the participant-specific available-item approach. Reporting both views preserves the fully matched five-item comparison while retaining partial baseline information where individual responses were missing.

For the eight complete cases, the mean baseline score was 3.38 out of 5 ($SD = 1.19$), compared with a post-session mean of 4.25 ($SD = 0.89$). The mean difference was 0.88 items ($SD = 0.83$), with observed differences ranging from 0 to 2.

The participant-specific available-item comparison retained all 12 participants by comparing each participant only on the repeated items for which a baseline response was available. Eight participants had all five repeated items available, two had four, one had three, and one had one. Across these matched subsets, the mean baseline percentage was 60.97% ($SD = 27.56$), the mean post-session percentage was 74.72% ($SD = 28.11$), and the mean percentage-point difference was 13.75 ($SD = 15.83$).

Table 6.5 and Figure 6.6 report the item-level matched comparison. Q11 increased from 5 correct responses among 11 matched cases at baseline (45.5%) to 8 of 11 post-session (72.7%), with three incorrect-to-correct transitions and no correct-to-incorrect transitions. Q13 increased from 6 of 10 matched cases (60.0%) to 9 of 10 (90.0%), again with three incorrect-to-correct transitions and no movement in the opposite direction. Q25 increased from 8 of 11 (72.7%) to 10 of 11 (90.9%); three

participants moved from incorrect to correct and one moved from correct to incorrect.

The aggregate percentage for Q12 remained at 72.7%, with the same eight participants answering correctly at both time points. Q14 also remained at 77.8% at the aggregate level, although the matched responses changed within the group: two participants moved from incorrect to correct and two moved from correct to incorrect. The transition counts are retained in Table 6.5 because the unchanged aggregate percentage for Q14 concealed movement in both directions.

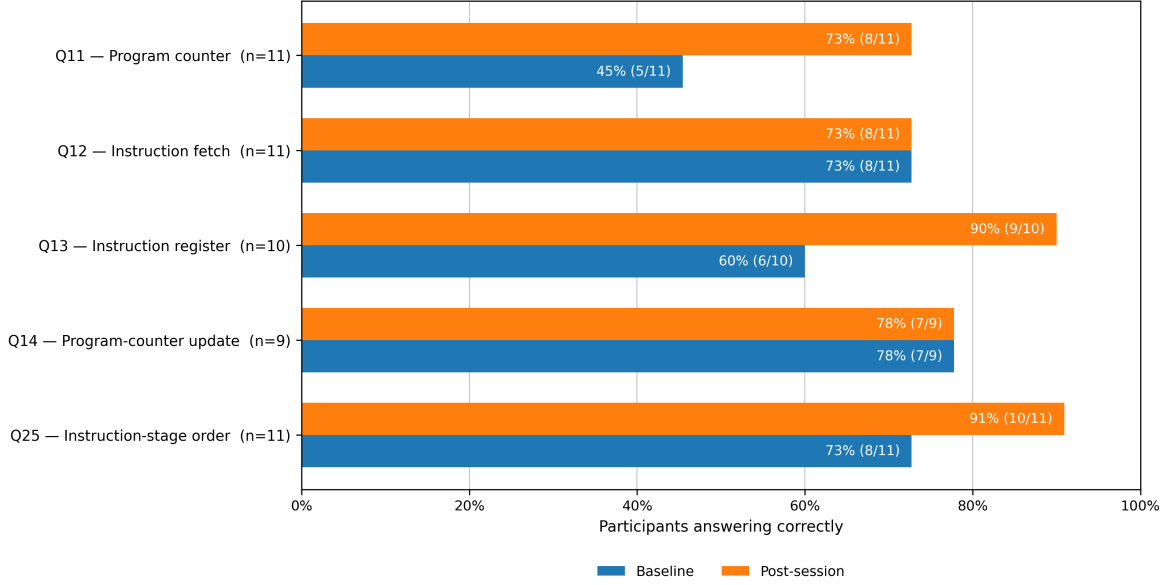


Figure 6.6: Matched baseline and post-session percentage-correct performance for the five repeated knowledge items. The comparison shows descriptive change across the overall learning session and should not be interpreted as an effect attributable to VR alone because the tailored refresher occurred between the two measurements.

Item	Matched n	Baseline correct	Post-session correct	Incorrect → Correct	Correct → Incorrect
Q11	11	5/11 (45.5%)	8/11 (72.7%)	3	0
Q12	11	8/11 (72.7%)	8/11 (72.7%)	0	0
Q13	10	6/10 (60.0%)	9/10 (90.0%)	3	0
Q14	9	7/9 (77.8%)	7/9 (77.8%)	2	2
Q25	11	8/11 (72.7%)	10/11 (90.9%)	3	1

Table 6.5: Matched baseline and post-session correctness for the five repeated knowledge items, including incorrect-to-correct and correct-to-incorrect transitions. The transition counts show participant-level movement that can be hidden when the aggregate percentage remains unchanged.

6.5 Qualitative Results

The final five questionnaire questions invited open-ended comments on what helped understanding, what remained confusing, what participants would change, whether the interface interfered with learning, and whether they would use the system again. These responses were considered alongside the anonymised researcher notes recorded during the sessions. Participant comments and researcher observations are reported separately because they represent different forms of evidence.

6.5.1 Open-Ended Questionnaire Feedback

Two forms of support appeared repeatedly in responses to the question asking what helped understanding most: direct interaction with the represented architecture and the local explanatory material provided around the lesson. One participant referred specifically to “the hands-on interactions such as picking up the registers and loading them”, while another described “the act of doing things” as the aspect that helped most. Similar responses referred to grabbing registers, the interactive structure of the application, simplified block representations, and the visibility of decode and execute stages. Guidance also appeared frequently in this part of the questionnaire. Six participants referred to instructions, notes, hints, or panels as useful parts of the experience, while one participant identified the explanation before entering VR as particularly important for following the later task.

The responses describing confusion and possible improvements were more varied. Several comments concerned the local interaction sequence rather than the overall route between the six lesson phases described in Chapters 4 and 5. One participant wanted clearer indication of where individual inputs should be placed, another participant described the control signals and some input locations as unclear, and one participant requested more explicit prompts showing “what to interact with, in what order, and what to do next”. Other comments focused on particular architecture tasks, including difficulties with register identification, ALU calculation, control-switch configuration, and the write-back stage. Decode in Practice mode also produced requests for additional visual support, including colour coding and a drawing or segmentation aid for the encoded instruction.

Information presentation and physical navigation formed another recurring group of comments. One participant described the three-panel arrangement as containing too much text to follow comfortably, while another requested clearer panel labels, less scrolling, and slower joystick movement. One commented on repeated movement back to the register area and on guidance arrows that were not always helpful in the reduced-support task. Another reported a visibility problem in which some text-input

areas were difficult to distinguish because the text and background were both light. These comments refer to the same panel, routing, and interaction systems described in Chapter 5, but record how those systems were experienced during the study.

Question 30 asked directly whether the VR interface had ever obstructed learning. Six participants described a specific interface or comfort-related problem, including nausea, dizziness, rotation and teleportation confusion, fast movement, object-placement difficulty, and text-input visibility. The remaining six answered No, Not really, Not at all, or otherwise stated that the interface had not interfered.

Responses concerning future use of the system varied in the conditions participants attached to continued use. Six participants gave an unqualified positive response, five described future use as conditional on factors such as additional polish, reduced discomfort, greater novice support, or some prior understanding of the topic, and one participant answered No because dizziness outweighed the perceived benefit. One participant described the system as useful when the learner already knows “a little bit about the topics” and wants to clarify concepts, while another questioned whether the current level of terminology and prior knowledge expected by the prototype would make it more suitable for learners who already partly understand the material.

6.5.2 Researcher Observations and Lesson Progression

The researcher notes recorded mode exposure, points of hesitation, support requirements, and comfort-related interruptions across the participant sessions. Since these notes were supplementary observational records rather than a formal completion log, the results are reported at group level where possible.

As shown in Table 6.6, all 12 participants attempted both Learning mode and Practice mode, while Test mode was attempted by two participants. Two Practice mode attempts were not completed. The amount of researcher support varied across the sessions, ranging from isolated navigation or interface assistance to more substantial guidance with task progression and interaction. Recurring points of difficulty included navigation and control handling, register interaction, control-signal and ALU-related reasoning, memory-stage behaviour, and write-back.

Comfort-related interruptions were also recorded during the sessions. Nausea, dizziness, or breaks associated with discomfort were explicitly noted for seven participants, with an additional case of movement-related disorientation. These observations are reported alongside the structured discomfort responses presented earlier in the chapter rather than being treated as a separate quantitative measure.

Table 6.7 summarises the instructions recorded across the three lesson modes. `add` was the most frequently encountered instruction, with 11 recorded attempts, followed by `addi` with six. Learning mode included the widest range of recorded instruc-

Recorded observation	Study record
Learning mode attempted	All 12 participants
Practice mode attempted	All 12 participants
Test mode attempted	2 participants
Explicitly unfinished Practice mode attempt	2 participants
Minimal researcher assistance recorded	2 participants
Substantial researcher assistance recorded	3 participants
Nausea, dizziness, or disorientation-related breaks recorded	7 participants
Additional movement-related disorientation recorded	1 participant
Recurring conceptual or interaction difficulties	Navigation and controls; register handling; control signals and ALU reasoning; memory-stage behaviour; write-back and task progression

Table 6.6: Summary of recorded lesson-mode exposure, incomplete attempts, researcher assistance, comfort-related interruptions, and recurring difficulties observed during the participant sessions. The table provides a group-level account of how far participants progressed and where additional support or interruption occurred.

Instruction	Learning	Practice	Test	Total recorded attempts
add	8	3	0	11
addi	2	4	0	6
sub	1	2	0	3
lw	2	0	1	3
sw	2	1	0	3
beq	1	0	1	2
and	1	0	0	1
or	0	1	0	1
slt	0	1	0	1

Table 6.7: Recorded instruction attempts across Learning mode, Practice mode, and Test mode during the participant sessions. The table shows how instruction exposure varied by mode and makes clear that later-mode performance was based on a smaller and less uniform set of instruction attempts.

tion exposure, while Practice mode contained one recorded instruction attempt per participant. Test mode exposure was limited to two recorded attempts, involving `lw` and `beq`, reflecting the uneven progression through the three modes across the study sessions.

6.6 Summary

The participant sample entered the study with varied prior architecture knowledge, VR experience, confidence, and topic familiarity. The post-session questionnaire recorded frequent agreement on the usability, engagement, and perceived-learning items, while the separate discomfort item was endorsed by nine participants. Post-session knowledge performance averaged 75.56% across Questions 11–25, with substantial variation between individual items. On the five repeated questions, the complete-case and available-item comparisons both recorded higher post-session descriptive averages than at baseline, alongside item-level cases in which performance remained unchanged or moved in both directions.

The open-ended responses and session notes added detail that was not captured by the structured measures. Participants repeatedly referred to hands-on interaction, visual representation, instructions, and hints when describing what helped them, while confusion and requested improvements concerned control signals, register and memory interactions, information density, local interaction sequencing, movement, and comfort. All 12 participants attempted Learning mode and Practice mode, two attempted Test mode, and the amount of researcher support recorded during the sessions ranged from minimal assistance to substantial guidance. Chapter 7 considers how these findings bear on the research question, the design choices established in Chapter 4, and the literature reviewed in Chapter 2.

Chapter 7

Discussion

This chapter interprets the findings reported in Chapter 6 in relation to the research question, the educational problem established earlier in the dissertation, and the design choices through which the prototype attempted to address it. The evaluation remains bounded by the exploratory study design described in Chapter 3: the sample was small, there was no non-VR control condition, the baseline probe was followed by a tailored refresher before headset use, and exposure to the later lesson modes was uneven. The findings therefore cannot establish that virtual reality caused improved understanding or that it is more effective than conventional teaching. They can, however, be used to judge whether this representation of single-cycle instruction execution was sufficiently usable, conceptually useful, and educationally coherent to justify its role as a supplementary learning environment.

The discussion follows the four evaluation aspects introduced in Chapter 3. The first concerns usability and clarity, including the extent to which interface or physical friction competed with the intended learning task. The second considers conceptual support by bringing together perceived learning, post-session knowledge performance, and the limited repeated baseline/post-session evidence. The third examines progression from guided to reduced support across Learning mode, Practice mode, and Test mode. The fourth considers how participants positioned the environment as a supplementary tool and what this implies for its most appropriate educational role. The four aspects are considered together because the educational usefulness of the environment depends on how they interact.

7.1 Usability, Clarity, and Interface Friction

The educational value of the prototype depends first on whether learners can understand and operate the environment well enough for the instructional structure to matter. A route intended to clarify instruction execution would be of limited use if

navigation, controls, or local interfaces became a greater source of confusion than the datapath itself. On this point, the results provide reasonable support for the basic structure of the environment: eleven of the 12 participants agreed or strongly agreed that they could navigate the environment without difficulty, that the controls felt intuitive, and that the visual presentation made the concepts easier to follow, while the six-item usability grouping produced a mean of 3.99. These responses suggest that the routed organisation described in Chapter 4 and implemented through stable lesson stations, gates, directional guidance, and repeated interaction patterns in Chapter 5 was broadly legible during the study.

Educational VR literature consistently treats usability as a prerequisite for the learning design to function effectively. Work on immersive computer-organisation learning keeps usability central to evaluation [13], while broader reviews of educational VR emphasise practical concerns around interaction design, deployment, accessibility, and the ability of learners to remain oriented within the experience [20, 22]. The results here indicate that the prototype generally cleared that initial threshold, with participants being able to follow the route, engage with the lesson stations, and progress into reduced-support use. This provides some support for the design decision to distribute instruction execution across stable world-space stations rather than recreating the full datapath as one dense three-dimensional object.

The more detailed feedback nevertheless shows that clarity was not uniform across the interface. The weakest positively framed usability item concerned the ability to focus on the learning task without distraction from the interface, for which only half of the sample selected Agree or Strongly Agree. Participants described the three-panel arrangement as text-heavy in places, requested clearer panel labels and reduced scrolling, and identified uncertainty around some interaction targets and input locations. Other comments concerned joystick speed, repeated movement between areas, guidance arrows that were not always useful during reduced-support work, and text-entry elements that were difficult to distinguish visually.

These findings closely reflect the information-design concerns discussed in Chapter 2. Immersive visualisation does not remove the problems of clutter, layout, or interaction complexity; it relocates them into a larger and more embodied space [21]. The text-heavy elements of the prototype also expose a more specific constraint of presenting instructional material in VR. Koch et al. note that even contemporary headsets remain poorly suited to scenarios requiring large quantities of reading and show that reading preferences can vary substantially with display characteristics [28]. The prototype deliberately avoided presenting the entire datapath at once, but the local interfaces could still become dense enough to compete for attention. The design strategy of dividing the machine into smaller spaces therefore appears useful at the

level of route organisation while still requiring restraint within each space. A future version would benefit less from adding further explanation than from making the existing explanation easier to scan, reducing unnecessary text, and making interaction targets and local task order more visually stable.

Physical comfort was a more substantial limitation than interface legibility. Nine participants agreed or strongly agreed that they experienced discomfort, motion sickness, or visual fatigue, while the researcher notes recorded nausea, dizziness, or disorientation-related breaks across several sessions. Furthermore, this often occurred alongside otherwise favourable responses to navigation and visual presentation, suggesting that while the participants could understand how to use the environment, experiencing enough physical strain interrupted or shortened their learning session.

The findings are consistent with broader concerns in educational VR, with reviews identifying discomfort, hardware readiness, accessibility, and practical deployment as persistent constraints on immersive learning [20, 22]. Dengel likewise cautions against assuming that greater immersion automatically produces stronger learning outcomes [19]. In this study, the representational advantages of VR and its physical costs appeared simultaneously, reinforcing those challenges. Because the study procedure placed participant welfare ahead of task completion, as required by the ethical boundaries established in Chapter 3, some participants stopped, paused, or progressed less far once discomfort accumulated. The later-mode evidence is consequently shaped not only by understanding or interface competence, but also by the amount of headset use each participant could comfortably sustain.

The participant background makes this issue harder to dismiss as simple inexperience with VR, as eleven of the 12 participants had previously used a headset, and nine described their level of VR experience as Good or High. Movement speed emerged as a recurring practical concern during the sessions, with teleportation used as an alternative where smooth locomotion became uncomfortable. This concern is consistent with Caputo et al., whose comparison of teleportation and joystick-based locomotion shows that navigation technique can affect simulator sickness, spatial orientation, presence, and the perceived difficulty of moving through a VR environment [29]. Although the sample is too small to draw conclusions about predictors of discomfort, the prevalence of these reports suggests that locomotion, movement speed, session duration, turning, and the physical organisation of the route should be treated as substantive design concerns in further development.

The usability evidence therefore supports the broad environmental strategy more strongly than the current implementation details. The route, phase separation, and repeated interaction logic appear clear enough to provide a workable teaching structure, while the interface and locomotion systems still impose avoidable costs. For

the educational purpose of the prototype, this shows that while spatial organisation seems capable of clarifying the lesson, learners must be able to remain comfortable and focused long enough to benefit from it.

7.2 Conceptual Support and Knowledge Outcomes

The central question is whether the experience offered conceptual support rather than only novelty, attention, or a visually distinctive way of presenting familiar information. The strongest participant-reported evidence concerning mental-model formation showed that all 12 participants agreed or strongly agreed that the visualisations helped them build a mental model of what was happening and that the system made abstract processes easier to understand. Eleven participants agreed or strongly agreed that they learned something useful from the session, while ten reported that the VR experience helped them understand the topic better.

Participants referred to handling registers, moving values through the environment, seeing stages locally, and being required to perform actions rather than only read about them. Several also identified the local instructions, notes, hints, and panels as important forms of support. These comments closely match the original educational problem described in Chapters 1 and 2.

Porter et al. show that procedural success in computer architecture does not necessarily imply secure conceptual understanding [5], while Yehezkel et al. emphasise the importance of helping learners construct a viable mental model of computer behaviour [6]. Broader work on VR in computer science education similarly identifies abstract and process-oriented topics as plausible uses of immersive visualisation [14–16]. Checa et al. provide a nearby example in hardware learning, where immersive presentation was especially relevant to understanding and making connections between concepts rather than simply recalling isolated facts [12]. The participant responses in this study fit that pattern more closely than a claim centred on memorisation.

The results also provide some support for that representational strategy, as participants repeatedly identified direct interaction and visible stages as useful, while the perceived-learning items were strongest on mental-model formation and the clarification of abstract processes. This emphasis on learner action is also consistent with Petersen et al., who show that immersion and interactivity can influence learning-related processes in VR [30]. Lee et al.’s account of immersive “visceralisation” is useful here because it describes how immersive representation can restore a more intuitive sense of relationships that abstraction has compressed [23]. In that limited sense, the prototype appears to have made parts of datapath execution easier to follow without abandoning the abstraction altogether.

The post-session knowledge check provides a more demanding test of how far that support extended. Overall performance was reasonably strong, with a mean of 11.33 correct answers out of 15, corresponding to 75.56%, so several questions were answered correctly by most or all participants. The execute-stage question was answered correctly by all 12 participants, while questions concerning the currently executing instruction, the fixed value of `$zero`, unconditional jump behaviour, and the broad order of instruction stages were each answered correctly by 11 participants. These results are consistent with several of the core ideas addressed by the prototype being understood after the session.

However, performance was not equally strong across all concepts, as the weakest item concerned the component primarily affected by a branch instruction, where only five participants selected the Program Counter and four instead selected the Instruction Register. The general question on identifying the instruction class that loads data from memory and the specific `sw` question were also weaker, with seven correct responses each. The store-word distractor was especially informative because four participants selected the response describing data moving from memory into a register, reversing the direction of the operation.

These errors point to areas where the prototype’s instructional translation may still be weaker than intended. Branch behaviour was given its own concluding Program Counter update phase precisely because next-PC reasoning can become compressed or implicit in a conventional datapath, yet the weak Q16 result suggests that recognising a branch or completing a branch-related action did not always produce a secure conceptual connection between the instruction and the Program Counter. Similarly, the memory results suggest that address calculation, memory location, and data-transfer direction did not become equally secure for all participants. These are not peripheral ideas to the design; they sit close to the learning outcomes around tracing values and predicting architectural state.

The fetch/decode distinction presents a similar case, as Q12 was answered correctly by eight participants and all four incorrect responses selected Decode rather than Fetch. The item also showed no aggregate change in the repeated baseline/post-session comparison. Because Fetch and Decode were physically separated in the environment, one possible reading is that the abstraction of much of Fetch into a simple carry-and-walk segment may have left the boundary between the two phases less explicit than intended. Future versions may therefore need to foreground the relationship between the neighbouring phases more clearly, rather than assuming that physical separation alone will establish the conceptual distinction.

The repeated baseline/post-session comparison offers additional evidence, but its methodological position is different. Among the eight complete cases, the mean

score across the five repeated questions increased from 3.38 to 4.25. The participant-specific available-item comparison showed a similar descriptive movement, from 60.97% at baseline to 74.72% after the session. At item level, Q11, Q13, and Q25 showed net increases, Q12 was unchanged, and Q14 retained the same aggregate percentage while individual participants moved in both directions.

These differences cannot be attributed to the VR environment alone, since the tailored refresher took place between the baseline probe and headset use, meaning that the post-session result reflects the overall learning sequence rather than an isolated VR intervention. Although this limitation is central to the interpretation, it does not make the comparison meaningless, as the repeated questions show that some of the targeted concepts were in a stronger descriptive position at the end of the combined session than at the beginning, while others remained stable or mixed. The pattern therefore supports a cautious account of concept reinforcement across the session rather than a causal claim of VR-driven improvement.

Participants were highly consistent in reporting that the environment helped them visualise and organise the process, whereas the objective concept evidence remained uneven. That combination of perceived learning and knowledge performance is more credible than treating either source in isolation. The prototype appears to have helped learners form and follow a representation of instruction execution, but one guided session did not eliminate every misconception. This distinction follows the concern raised by Porter et al. that conceptual understanding cannot be assumed from successful performance on selected tasks alone [5]. The evidence therefore supports conceptual assistance within the intended scope, while leaving clear room for stronger reinforcement of branch behaviour, memory direction, and the distinction between adjacent phases.

7.3 Progression from Guided to Reduced Support

The three-mode structure was designed to move the learner from explanation toward increasingly independent reconstruction. Learning mode provided the fullest local guidance, Practice mode reduced that support and required more deliberate decoding and decision-making, and Test mode retained the same route while removing most remaining assistance. The design follows a pattern already visible in architecture-facing educational systems, where tutorials or step-by-step support are provided before learners are expected to work with less guidance [7, 8]. Similar use of prompts and staged support appears across the wider VR-in-computer-science literature [16].

The study provides partial evidence that this progression functioned as intended, since all 12 participants attempted both Learning mode and Practice mode. The

transition into Practice mode therefore represented a genuine reduction in support rather than confinement to the most explanatory version of the lesson. At the same time, two Practice mode attempts were not completed, only two participants attempted Test mode, and the researcher notes recorded support needs ranging from isolated intervention to substantial assistance. The progression evidence should therefore be interpreted as successful movement into reduced guidance rather than as evidence of reliable independent completion. In the terms of the final learning outcome established in Section 4.1, every participant reached the first reduced-support condition, but the data remain too uneven to show whether most participants could reconstruct the entire lesson independently.

The observation notes help explain where the transition became difficult, with hesitation occurring around navigation and controls, register handling, ALU and control-signal reasoning, memory-stage behaviour, write-back, and the broader sequencing of what to do next. Participants requested clearer prompts, better indication of where individual inputs should be placed, colour coding, a drawing or segmentation aid during Decode in Practice mode, and clearer organisation of the information provided by the local panels. This reveals an important distinction between conceptual and interaction scaffolding. Practice mode is intended to reduce help with the architecture problem by asking the learner to remember how to decode an instruction, identify the necessary operands, and make the relevant control-facing decisions. It is less useful if difficulty arises because the learner cannot determine which object to pick up, where to place an input, or how to navigate to the next station, since the mode then begins to test familiarity with the interface rather than independence in architecture reasoning.

A future revision should therefore preserve stable interaction conventions while withdrawing conceptual explanation more selectively. The local mechanics of selecting, placing, validating, and progressing should remain predictable across modes, while hints, reference material, worked explanation, and tolerance for conceptual errors can be reduced. This would make failure in Practice or Test more interpretable: a learner who cannot continue would be more likely to be revealing a gap in architectural reasoning rather than a gap in interface memory.

Tan et al.’s architecture VR system is relevant to this point because its support is provided step by step before being reduced in later tasks [7]. The principle is similar to the current three-mode structure, but the study findings show that the timing and form of support withdrawal require careful calibration. Guidance needs to recede far enough for the learner to reconstruct the reasoning, while the underlying interaction language of the environment remains sufficiently stable that it does not become a second assessment target.

The limited Test mode exposure further constrains the interpretation of inde-

pendence. Although the mode was intended to provide the clearest indication that a learner could complete the route with minimal assistance, only two participants attempted it, and session duration, discomfort, background, and earlier support needs all affected whether it was reached. The small number of Test attempts should therefore be treated as insufficient evidence rather than as evidence of incapacity.

Instruction exposure also varied across participants due to time constraints and participant understanding. The session records include arithmetic, immediate, memory, and branch instructions, with `add` and `addi` encountered most often. That breadth helped exercise different regions of the prototype, but it also means that reduced-support performance was not based on one identical task. An arithmetic instruction does not place the same demands on memory, control flow, or write-back reasoning as a load, store, or branch. A later study intended to compare progression directly would benefit from a standardised mode sequence and a smaller common instruction set before introducing additional variation.

Even with these limitations, the three-mode architecture created a meaningful distinction between being taught the route and being asked to reconstruct it, and all participants reached the point at which that distinction became relevant. The evidence supports the progression model as a pedagogical structure more strongly than it supports Test mode as an assessment mechanism. Further work should concentrate on making the reduction of support cleaner, standardising progression where possible, and ensuring that comfort and interface demands do not determine who reaches the later modes.

7.4 Value as a Supplementary Educational Tool

The participant evidence supports the supplementary role established throughout the dissertation more strongly than any claim that the environment should replace conventional teaching. Nine participants agreed or strongly agreed that they would prefer the system as a supplement to traditional teaching materials, while the open-ended responses also showed that future use was often tied to factors such as prior familiarity, stronger novice support, reduced discomfort, or further polish.

These qualifications are consistent with the pedagogical position established in Chapters 2 and 4. Lectures, tutorials, diagrams, simulators, and worked examples already provide terminology, structure, reference material, and repeated practice; the prototype was instead designed for the point at which a learner has encountered those representations but still struggles to connect them into a stable account of instruction execution. This also matches broader work that frames immersive tools as complementary support rather than replacements for conventional teaching [16, 18, 19]. The

participant responses therefore place the prototype most credibly as a bridge between prior exposure to the datapath and a more stable learner-facing account of how instruction execution unfolds.

7.5 Study and Prototype Limitations

The interpretation of the study is limited first by the size, composition, and structure of the evaluation. Only twelve participants, recruited through voluntary convenience sampling, completed the study, and the sample combined different starting points rather than representing a clearly defined novice cohort. Nine participants had previously studied computer architecture or computer organisation, while eleven had previously used a VR headset. The study design also does not permit causal attribution: there was no non-VR comparison condition, and the repeated baseline questions were followed by a tailored verbal refresher before participants entered the headset. Any baseline/post-session difference may therefore reflect the refresher, the VR experience, their combination, or other effects of the overall learning session. Long-term retention was not measured, as the post-session knowledge check was completed immediately after the VR experience.

The evidence is also uneven across the different parts of the lesson. All participants attempted Learning and Practice, but only two attempted Test, and the amount of researcher intervention varied across sessions. The study therefore provides stronger evidence about guided use and initial transition into reduced support than about consistent independent completion. Physical discomfort further affected the study conditions: nine participants endorsed the discomfort item, and the session notes recorded multiple breaks caused by nausea, dizziness, or disorientation. This reduced the amount of comparable exposure across participants and indicates that the current locomotion and route design are not yet suitable for assuming uninterrupted use. The interface introduced additional cognitive burden in places through text density, scrolling, visibility problems, unclear interaction sequencing, and uncertainty over some input locations. Because the system was intended to reduce the burden of following a dense datapath representation, these issues risked reintroducing some of that difficulty through the interface itself.

The implemented architecture scope was also deliberately narrow. The prototype does not evaluate multicycle execution, pipelining, hazards, cache behaviour, or a complete MIPS instruction set. This boundary allowed the design to concentrate on a coherent lesson loop, but it limits transfer of the findings to more complex architecture topics and leaves the value of the pedagogical-serialisation strategy beyond single-cycle execution unresolved. The researcher observations were also supplemen-

tary session notes rather than a formal behavioural observation protocol. They were useful for identifying mode exposure, points of hesitation, intervention, and comfort-related interruption, but they do not support fine-grained quantitative claims about behaviour frequency. These limitations bound the claims made here while still allowing the study to judge whether the design direction warrants further development and more controlled evaluation.

7.6 Answer to the Research Question

The research question asked:

To what extent can an immersive virtual reality learning environment support students' understanding of selected single-cycle MIPS datapath concepts as a supplementary educational tool?

Within the scope of this evaluation, the findings support a qualified positive answer. The prototype appears capable of providing useful supplementary support for selected single-cycle datapath concepts, particularly by making instruction execution easier to follow as a sequence of visible and interactive relationships. Participants consistently reported that the visualisations helped them form a mental model and made abstract processes easier to understand, while the open-ended responses repeatedly identified hands-on interaction, visible stages, and local guidance as useful parts of the experience. Post-session knowledge performance was generally strong on several targeted concepts, and the repeated baseline/post-session comparison showed descriptive strengthening on part of the repeated item set after the overall learning session.

The extent of that support was nevertheless uneven. Branch-to-PC reasoning, memory-transfer direction, and some stage distinctions remained difficult for part of the sample. Movement from Learning into Practice occurred across all participants, but independent progression beyond Practice was not observed consistently enough to establish consistent independent performance. Interface ambiguity and physical discomfort also reduced the amount of effective exposure available to some participants. The study therefore supports conceptual assistance and guided progression more strongly than it supports claims of mastery or independent transfer.

The findings also reinforce the supplementary position established throughout the dissertation. The environment is best supported as an additional representation of instruction execution alongside lectures, diagrams, tutorials, and simulators. Its main contribution is the design and exploratory evaluation of a pedagogical translation in which hidden and effectively concurrent datapath behaviour is reorganised into sequential teaching phases that can be seen, followed, acted upon, and progressively

reconstructed with less support. The evidence is sufficient to justify further development of that design direction, while the limitations of the study define the questions that would need to be addressed before stronger educational claims could be made.

7.7 Summary

This chapter interpreted the study findings across usability and clarity, conceptual support, progression from guided to reduced support, and the prototype's value as a supplementary educational tool. Across these areas, the evidence supports a qualified view of the learner-facing translation of single-cycle datapath reasoning, while interface friction, physical discomfort, uneven later-mode exposure, and the exploratory study design constrain the strength of that conclusion. Chapter 8 returns to the original educational problem, consolidates the contribution of the dissertation, and outlines the further work needed before broader claims can be made.

Chapter 8

Conclusion

Computer architecture requires learners to connect several representations of instruction execution that are often introduced separately: instruction formats, registers, control decisions, datapath structure, memory behaviour, and changes to architectural state. As established in Chapter 2, difficulty can persist even when the individual components are familiar, because the learner must still turn those partial representations into a coherent account of what the processor is doing [5, 6]. This dissertation addressed that narrower educational problem through a virtual reality learning environment focused on selected single-cycle MIPS datapath concepts and positioned throughout as a supplement to conventional teaching.

The resulting prototype was designed around guided instruction tracing rather than around a literal three-dimensional reconstruction of the processor. Selected datapath relationships were redistributed across six learner-facing phases: Fetch, Decode, Execute, Memory, Write-Back, and Program Counter Update. This structure was supported through physical instruction handling, register interaction, visible value transfer, local validation, and progressively reduced guidance across Learning mode, Practice mode, and Test mode. The prototype was then evaluated through the exploratory participant study described in Chapter 3, with the findings reported in Chapter 6 and interpreted in Chapter 7.

8.1 Main Contribution and Research Question

The main contribution lies in the way the single-cycle datapath was translated for teaching. Hardware behaviour that is compact, partly hidden, and effectively concurrent was reorganised into a sequential reasoning path that learners could see, follow, act upon, and revisit. The aim was not to reproduce processor timing literally, but to preserve the relationships that a learner must understand while making those relationships more legible during instruction. The participant evidence provides some support

for this design choice: visualisation, mental-model formation, direct interaction, and local guidance were repeatedly identified as useful, while the study also exposed where the translation remained weaker, particularly around some control-flow, memory, and phase-boundary concepts.

Within the limits of the evaluation, the research question receives a qualified positive answer. The prototype appears capable of supporting selected single-cycle MIPS datapath concepts as a supplementary learning environment, with its strongest value lying in the visual and interactive organisation of instruction execution into a learner-facing sequence. Participants consistently reported support for mental-model formation and understanding of abstract processes, and all participants progressed from Learning mode into Practice mode. That evidence remains limited by the small sample, the absence of a non-VR comparison condition, the tailored refresher delivered between the baseline probe and headset use, the limited progression into Test mode, the lack of delayed retention measurement, and the interface or comfort problems experienced by some participants. The findings therefore support further development of this design approach without demonstrating that VR is superior to established forms of computer-architecture teaching.

The prototype’s most credible educational role is as an additional representation used after learners have encountered the terminology and structure of the datapath. In that position, it can slow instruction execution down and make selected relationships visible without replacing the lectures, diagrams, tutorials, and simulators that establish the wider subject context [16, 19].

8.2 Future Work

The most immediate future work follows from the participant study itself. A stronger evaluation would use a larger and more clearly defined participant sample, ideally involving students currently taking a computer architecture module, together with a comparative condition using an alternative teaching format. A more standardised lesson sequence would also make progression across Learning mode, Practice mode, and Test mode easier to interpret, particularly if students encountered a common set of instructions before later variation was introduced. Delayed assessment would then be needed to determine whether the mental-model and concept support reported immediately after the session persisted over time. The observational component could likewise be strengthened through a predefined protocol that distinguishes conceptual assistance from navigation or interface assistance.

The second priority is refinement of the current VR experience. Comfort should be addressed directly through greater control over locomotion speed, reduced reliance

on sustained smooth movement, clearer alternatives such as teleportation, and route design that limits unnecessary travel between stations. The interface should also be simplified where the study identified text density, scrolling, uncertain task order, or unclear interaction targets. Particular attention should be given to preserving stable interaction conventions as guidance is reduced, so that Practice and Test modes assess architectural reasoning rather than memory for the interface. Concepts that remained weaker in the study, including branch-to-PC reasoning, memory-transfer direction, and the relationship between neighbouring phases such as Fetch and Decode, should also receive stronger instructional reinforcement.

Beyond those study-led refinements, broader technical expansion becomes a longer-term priority. The supported instruction set could first be widened so that more arithmetic, memory, and control-flow cases can be explored without changing the underlying lesson model. A later version could then investigate whether the same pedagogical-serialisation approach remains useful for multicycle execution or pipelined behaviour, following the progression students would normally encounter as a computer architecture course moves from basic instruction execution toward stalls, hazards, and overlapping instruction flow. These extensions would not simply add more content; they would test whether the central design principle developed here continues to hold once the processor can no longer be explained through one bounded single-cycle walk-through.

The dissertation therefore establishes a bounded design contribution: a sequential, interactive translation of selected single-cycle MIPS datapath reasoning that can be used as supplementary support for instruction tracing. The exploratory study provides enough evidence to justify further refinement while also showing that clarity, comfort, support withdrawal, and scope remain central constraints. Comparative and longitudinal evaluation is now needed to determine how far the approach extends beyond the prototype and study reported here.

Bibliography

- [1] Trinity College Dublin. Computer science – modules, 2026. URL <https://www.tcd.ie/courses/undergraduate/modules/computer-science/>. Undergraduate module listing for the Computer Science programme.
- [2] Trinity College Dublin. CSU22022 – computer architecture i, 2026. URL <https://teaching.scss.tcd.ie/module/csu22022-computer-architecture-i/>. Module coordinator: Prof. Michael Manzke.
- [3] University of Arizona. BS and BA in computer science undergraduate program curriculum structure, 2026. URL <https://cs.arizona.edu/undergraduate/undergraduate-programs/comp-sci-undergrad-program-structure>. Undergraduate curriculum structure for Computer Science.
- [4] University of Arizona. CSC252 – computer organization, 2026. URL <https://catalog.arizona.edu/courses/0097831>. Official course catalogue description.
- [5] Leo Porter, Saturnino Garcia, Hung-Wei Tseng, and Daniel Zingaro. Evaluating student understanding of core concepts in computer architecture. In *Proceedings of the 18th ACM Conference on Innovation and Technology in Computer Science Education*, pages 279–284. Association for Computing Machinery, 2013. ISBN 978-1-4503-2078-8. doi: 10.1145/2462476.2462490.
- [6] Cecile Yehezkel, Mordechai Ben-Ari, and Tommy Dreyfus. The contribution of visualization to learning computer architecture. *Computer Science Education*, 17(2):117–127, 2007. ISSN 1744-5175. doi: 10.1080/08993400601165545.
- [7] Kai Hwee Alvin Tan, Yiyu Cai, Bheema Thiagarajan Lokesh, Yongqing Zhu, Chengzhi Su, and Qi Cao. VR serious game for learning the computer organisation and architecture course. In *2023 3rd International Conference on Educational Technology (ICET)*, pages 1–6. IEEE, 2023. ISBN 979-8-3503-0233-2. doi: 10.1109/ICET59358.2023.10424233.
- [8] Ahmed Tlili, Fathi Essalmi, and Mohamed Jemni. Improving learning computer

- architecture through an educational mobile game. *Smart Learning Environments*, 3(1):7, 2016. ISSN 2196-7091. doi: 10.1186/s40561-016-0030-6.
- [9] Jyotiprakash Mishra. MIPS single-cycle datapath, 2024. URL <https://blog.jyotiprakash.org/mips-single-cycle-datapath>. Blog post. Available at: <https://blog.jyotiprakash.org/mips-single-cycle-datapath>.
- [10] Hiroaki Fukuda, Paul Leger, and Ismael Figueroa. A practical methodology to learn computer architecture, assembly language, and operating system. In *Proceedings of the 12th International Conference on Computer Supported Education*, volume 1, pages 333–340. INSTICC, SciTePress, 2020. ISBN 978-989-758-417-6. doi: 10.5220/0009319503330340.
- [11] Maria-Iuliana Dascalu, Sergean Bagis, Melania Nitu, Oana-Maria Ferche, and Alin Dragos Bogdan Moldoveanu. Experiential learning VR system for studying computer architecture. *Romanian Journal of Human-Computer Interaction*, 10(3):197–215, 2017. ISSN 1843-4460.
- [12] David Checa, Ines Miguel-Alonso, and Andres Bustillo. Immersive virtual-reality computer-assembly serious game to enhance autonomous learning. *Virtual Reality*, 27(4):3301–3318, 2023. ISSN 1434-9957. doi: 10.1007/s10055-021-00607-1.
- [13] Mariam Ali Alnuaimi and Mamoun Awad. VR environment of digital design laboratory: A usability study. *Frontiers in Virtual Reality*, 6:1566680, 2025. ISSN 2673-4192. doi: 10.3389/frvir.2025.1566680.
- [14] Johanna Pirker, Andreas Dengel, Michael Holly, and Saeed Safikhani. Virtual reality in computer science education: A systematic review. In *Proceedings of the 26th ACM Symposium on Virtual Reality Software and Technology*. Association for Computing Machinery, 2020. ISBN 978-1-4503-7619-8. doi: 10.1145/3385956.3418947.
- [15] Friday Joseph Agbo, Ismaila Temitayo Sanusi, Solomon Sunday Oyelere, and Jarkko Suhonen. Application of virtual reality in computer science education: A systemic review based on bibliometric and content analysis methods. *Education Sciences*, 11(3):142, 2021. ISSN 2227-7102. doi: 10.3390/educsci11030142.
- [16] Azin S. Janani, Yanzhuo Yang, Xundi Lu, Hai-Ning Liang, and Nilufar Baghaei. Virtual reality applications in computer science education: Scoping review. *Computers & Education: X Reality*, 8:100159, 2026. ISSN 2949-6780. doi: 10.1016/j.cexr.2026.100159.

- [17] Johanna Pirker, Johannes Kopf, Alexander Kainz, Andreas Dengel, and Benjamin Buchbauer. The potential of virtual reality for computer science education—engaging students through immersive visualizations. In *2021 IEEE conference on virtual reality and 3D user interfaces abstracts and workshops (VRW)*, pages 297–302. IEEE, 2021. ISBN 978-1-6654-4057-8. doi: 10.1109/VRW52623.2021.00060.
- [18] Abdulrazak Yahya Saleh, Goh Suk Chin, Mohd Kamal Othman, Fitri Suraya Mohamad, and Chwen Jen Chen. Immersive visualization of Python coding using virtual reality. *International Journal on Advanced Science, Engineering and Information Technology*, 13(1):336–347, feb 2023. ISSN 2088-5334. doi: 10.18517/ijaseit.13.1.16028.
- [19] Andreas Dengel. How important is immersion for learning in computer science replugged games? In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, pages 1165–1171. Association for Computing Machinery, 2020. ISBN 978-1-4503-6793-6. doi: 10.1145/3328778.3366837.
- [20] Matt Cook, Zack Lischer-Katz, Nathan Hall, Juliet Hardesty, Jennifer Johnson, Robert McDonald, and Tara Carlisle. Challenges and strategies for educational virtual reality: Results of an expert-led forum on 3D/VR technologies across academic institutions. *Information Technology and Libraries*, 38(4):25–48, 2019. ISSN 2163-5226. doi: 10.6017/ital.v38i4.11075.
- [21] Elif Hilal Korkut and Elif Surer. Visualization in virtual reality: A systematic review. *Virtual Reality*, 27(2):1447–1480, 2023. ISSN 1434-9957. doi: 10.1007/s10055-023-00753-8.
- [22] Agariadne Dwinggo Samala, Soha Rawas, Sandi Rahmadika, Santiago Criollo-C, Ryan Fikri, and Randi Proska Sandra. Virtual reality in education: Global trends, challenges, and impacts—game changer or passing trend? *Discover Education*, 4(1):229, 2025. ISSN 2731-5525. doi: 10.1007/s44217-025-00650-z.
- [23] Benjamin Lee, Dave Brown, Bongshin Lee, Christophe Hurter, Steven Drucker, and Tim Dwyer. Data visceralization: Enabling deeper understanding of data using virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1095–1105, 2021. ISSN 1941-0506. doi: 10.1109/TVCG.2020.3030435.
- [24] Unity Technologies. XR interaction toolkit 3.4 manual, 2026. URL <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.4/manual/index.html>. Official package manual.

- [25] Unity Technologies. VR template 9.2 manual, 2026. URL <https://docs.unity3d.com/Packages/com.unity.template.vr@9.2/manual/index.html>. Official package manual.
- [26] Cuboom. Free LowPoly SciFi. Unity Asset Store, feb 2018. URL <https://assetstore.unity.com/packages/3d/environments/sci-fi/free-lowpoly-scifi-110070>. Released 8 February 2018.
- [27] Oiva Kahri. 3D Sprite Sci-Fi Props. Unity Asset Store, jun 2023. URL <https://assetstore.unity.com/packages/3d/props/3d-sprite-sci-fi-props-246349>. Released 13 June 2023.
- [28] Daniel Koch, Alan Medlar, and Dorota Glowacka. Read the fine print: Assessing the user experience of reading in high resolution virtual reality. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2024. ISBN 979-8-4007-0331-7. doi: 10.1145/3613905.3650916.
- [29] Ariel Caputo, Massimo Zancanaro, and Andrea Giachetti. Eyes on teleporting: Comparing locomotion techniques in virtual reality with respect to presence, sickness and spatial orientation. In *Human-Computer Interaction – INTERACT 2023*, volume 14144 of *Lecture Notes in Computer Science*, pages 547–566. Springer, 2023. ISBN 978-3-031-42286-7. doi: 10.1007/978-3-031-42286-7_31.
- [30] Gustav Bøg Petersen, Giorgos Petkakis, and Guido Makransky. A study of how immersion and interactivity drive VR learning. *Computers & Education*, 179: 104429, 2022. ISSN 0360-1315. doi: 10.1016/j.compedu.2021.104429.

Appendix A

Participant Questionnaire

This appendix reproduces the final participant-facing questionnaire used during live data collection.

Participant ID: _____

Date: _____

Screening and Background

1. Are you 18 or older?

☐ Yes ☐ No

2. Have you previously studied computer architecture or computer organisation?

☐ Yes ☐ No

3. Have you used a VR headset before?

☐ Yes ☐ No

4. How experienced are you with Virtual Reality?

☐ None ☐ Limited ☐ Moderate ☐ Good ☐ High

5. Before this session, how confident are you in your understanding of computer architecture concepts?

☐ Extremely confident ☐ Somewhat confident ☐ Neutral ☐ Somewhat not confident
☐ Extremely not confident

6. How difficult do you usually find topics such as instruction flow, pipelining, stack behaviour, and memory hierarchy?

☐ Extremely difficult ☐ Somewhat difficult ☐ Neutral ☐ Somewhat easy ☐ Extremely easy

7. Which of the following topics have you previously studied?

Select all that apply.

- | | | |
|---|-------------------------------------|--|
| ☐ Instruction Flow | ☐ Pipelining | ☐ Stack |
| ☐ Memory Hierarchy | ☐ Assembly | ☐ None of the above |

System Experience

8. Please indicate how strongly you agree with the following statements for System Usability.

Response options: SD = Strongly Disagree, D = Disagree, N = Neutral, A = Agree, SA = Strongly Agree.

	SD	D	N	A	SA
The VR system was easy to learn.	☐	☐	☐	☐	☐
I could navigate the environment without difficulty.	☐	☐	☐	☐	☐
The controls felt intuitive.	☐	☐	☐	☐	☐
The visual presentation made the concepts easier to follow.	☐	☐	☐	☐	☐
I could focus on the learning task without being distracted by the interface.	☐	☐	☐	☐	☐
I would feel confident using this system again.	☐	☐	☐	☐	☐
I experienced discomfort, motion sickness, or visual fatigue while using the system.	☐	☐	☐	☐	☐

9. Please indicate how strongly you agree with the following statements for System Engagement.

Response options: SD = Strongly Disagree, D = Disagree, N = Neutral, A = Agree, SA = Strongly Agree.

	SD	D	N	A	SA
The experience held my attention.	☐	☐	☐	☐	☐
I felt actively involved in the learning process.	☐	☐	☐	☐	☐
The system made the topic feel more interesting.	☐	☐	☐	☐	☐
I was motivated to explore the concepts further.	☐	☐	☐	☐	☐
The interactive format improved my focus compared with standard learning materials.	☐	☐	☐	☐	☐

10. Please indicate how strongly you agree with the following statements about your learning experience.

Response options: SD = Strongly Disagree, D = Disagree, N = Neutral, A = Agree, SA = Strongly Agree.

	SD	D	N	A	SA
The VR experience helped me understand the topic better.	☐	☐	☐	☐	☐
The visualisations helped me build a mental model of what was happening.	☐	☐	☐	☐	☐
The system made abstract processes easier to understand.	☐	☐	☐	☐	☐
I would prefer this as a supplement to traditional teaching materials.	☐	☐	☐	☐	☐
I believe I learned something useful from this session.	☐	☐	☐	☐	☐

Knowledge Check

Answer the following questions to assess your understanding of the computer architecture concepts covered in the session.

Reference Guide

- **MIPS**: Microprocessor without Interlocked Pipeline Stages, a Reduced Instruction Set Computer (RISC) architecture commonly used for teaching computer architecture.
- **add**: Adds the values from two registers and stores the result in a destination register.
- **addi**: Adds an immediate (constant) value to a register and stores the result.
- **lw** (Load Word): Loads a 32-bit word from memory into a register.
- **sw** (Store Word): Stores a 32-bit word from a register into memory.
- **beq** (Branch if Equal): Branches to a target address if two register values are equal.
- **j** (Jump): Unconditionally jumps to a specified instruction address.

11. Which register stores the address of the next instruction?

- | | |
|---|--|
| ☐ Stack Pointer | ☐ Program Counter |
| ☐ Instruction Register | ☐ Base Register |

12. Which stage of the instruction cycle reads an instruction from memory?

- | | |
|----------------------------------|-------------------------------------|
| ☐ Decode | ☐ Fetch |
| ☐ Execute | ☐ Write-back |

13. During instruction execution, the instruction register primarily stores:

- | | |
|--|--|
| ☐ Program variables | ☐ The currently executing instruction |
| ☐ The stack pointer value | ☐ Cache addresses |

14. What happens to the program counter after an instruction is fetched?

- | | |
|---|--|
| ☐ It resets to zero | ☐ It increments to the next instruction address |
| ☐ It copies into the stack pointer | ☐ It stores the result of the ALU |

15. Which pipeline stage usually performs arithmetic operations?

- | | |
|----------------------------------|-------------------------------------|
| ☐ Fetch | ☐ Decode |
| ☐ Execute | ☐ Write-back |

16. Branch instructions primarily affect which component?

- | | |
|--|---|
| ☐ Cache memory | ☐ Program counter |
| ☐ Stack pointer | ☐ Instruction register |

17. What does this instruction do?

`add $t0, $t1, $t2`

- ☐ Adds \$t1 and \$t2 and stores the result in \$t0
- ☐ Adds \$t0 and \$t1 and stores the result in \$t2
- ☐ Moves \$t1 into \$t2
- ☐ Multiplies registers

18. What does this instruction do?

`addi $t0, $zero, 10`

- ☐ Stores 10 into \$t0
- ☐ Clears \$t0
- ☐ Moves \$t0 into \$zero
- ☐ Adds two registers

19. Which instruction type loads data from memory into a register?

- | | |
|-------------------------------|--------------------------------|
| ☐ ADD | ☐ STORE |
| ☐ LOAD | ☐ JUMP |

20. What is the role of register \$zero in MIPS?

- | | |
|---|--|
| ☐ Stores the program counter | ☐ Always contains the value 0 |
| ☐ Stores return addresses | ☐ Tracks the stack pointer |

21. What does this instruction do?

lw \$t0, 0(\$s0)

- ☐ Stores \$t0 into memory
- ☐ Loads a word from memory into \$t0
- ☐ Moves \$s0 into \$t0
- ☐ Clears memory

22. What does this instruction do?

sw \$t0, 4(\$s0)

- ☐ Value from \$t0 is stored in memory
- ☐ Value from memory is loaded into \$t0
- ☐ \$s0 is overwritten
- ☐ Stack pointer moves

23. What does this instruction do?

beq \$t0, \$t1, label

- ☐ Jump if \$t0 > \$t1
- ☐ Jump if \$t0 == \$t1
- ☐ Jump if \$t0 < \$t1
- ☐ Always jump

24. What does this instruction do?

j label

- ☐ Jumps to label unconditionally
- ☐ Jumps only if two registers are equal
- ☐ Stores the return address in \$ra
- ☐ Loads a value from memory

25. Put the instruction stages in the correct order.

- | | |
|---|---|
| ☐ Execute, Fetch, Decode, Write-back | ☐ Fetch, Decode, Execute, Write-back |
| ☐ Decode, Fetch, Execute, Write-back | ☐ Fetch, Execute, Decode, Write-back |

26. How confident are you in your answers to the previous section?

- ☐ Not at all confident ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 Very confident

Open-Ended Feedback

27. What part of the experience helped your understanding most?

28. What part was confusing or ineffective?

29. What would you improve in the system?

30. Were there any moments where the VR interface got in the way of learning?

31. Would you use this again as a learning aid? Why or why not?

Appendix B

Participant Information Leaflet

This appendix reproduces the Participant Information Leaflet used during recruitment and consent for the live study.

Researcher: Priyansh Nayak

Programme: MSc Computer Science - AVR

School: School of Computer Science & Statistics, Trinity College Dublin

Supervisor: Gareth Young

Invitation

You are being invited to take part in a research study about the use of virtual reality for teaching computer architecture concepts. Before you decide whether to take part, it is important that you understand why the research is being carried out and what your participation would involve. Please read the following information carefully and ask any questions you may have before deciding.

What is the purpose of the study?

Computer architecture is often considered difficult to learn because many of its core processes are abstract and not directly visible. This study aims to evaluate whether a virtual reality learning environment can help support understanding of concepts such as instruction flow, pipelining, memory hierarchy, and stack behaviour. The study forms part of a Master's dissertation in Computer Science at Trinity College Dublin.

Why have I been invited to take part?

You are being invited because the study is seeking adult participants who are willing to take part in an evaluation of a virtual reality learning environment. Participants must be aged 18 or over and able to provide informed consent.

Do I have to take part?

No. Participation is entirely voluntary. If you decide to take part, you will be asked to

sign a consent form before the study begins. You may withdraw at any time without penalty. If you choose to withdraw before your data has been anonymised and included in the analysis, your identifiable or linkable data will be deleted where possible.

What will happen if I take part?

If you agree to take part, you will attend one study session lasting approximately 45 to 60 minutes. During the session:

1. you will be briefed on the study and asked to provide written informed consent;
2. you will use a prototype virtual reality learning environment;
3. you will complete a small number of guided learning tasks;
4. you will complete short questionnaires on conceptual understanding, engagement, and usability;
5. you may also be invited to provide brief written open-ended feedback on your experience.

Are there any risks in taking part?

There is a low risk of temporary physical discomfort from using the VR headset, including eye strain, dizziness, or motion sickness. If you feel any discomfort at any point, the session will stop immediately and you may remove the headset straight away. You may pause or withdraw from the study at any time without penalty.

Are there any benefits to taking part?

There is no direct benefit to you from taking part. However, you may gain some insight into the topic being studied, and your participation will contribute to research on virtual reality as a learning tool in computer science education.

Will my taking part in this study be kept confidential?

Any personal data collected for study administration, such as contact details used for arranging participation, will be kept separate from the research data. Research responses will be anonymised as early as practicable for analysis and reporting. Only the researcher and supervisor, where necessary, will have access to identifiable study administration data.

What personal data will be collected?

The study may collect limited personal data for administration purposes, such as your name and email address if needed for arranging participation. Research data will include questionnaire responses and written feedback. No special category personal data

is intended to be collected for this study. The study uses a Meta VR headset provided to the researcher by the School of Computer Science & Statistics. Participants will not be required to use their own Meta account. The research team will not intentionally collect Meta account data as part of the study. However, as third-party hardware and software are used, limited device- or platform-level processing outside the control of the research team may occur.

How will my data be stored and protected?

Personal data and study data will be stored on password-protected devices and university-approved storage systems. Identifiable administration data will be stored separately from research responses. Devices used for storage and access will be protected by password security, encryption, and up-to-date anti-virus protection.

How long will my data be kept?

Personal data held in identifiable or coded form, such as contact details used for study administration, will be retained for up to 12 months and then deleted. Research data will be anonymised as early as practicable for analysis and reporting.

What will happen to the results of the study?

The results of the study will be used for a Master's dissertation and may also be presented in academic work arising from the project. Any reporting of the findings will be done in anonymised form so that individual participants are not identified.

Who is organising and funding the study?

The study is being conducted by Priyansh Nayak as part of a Master's dissertation at Trinity College Dublin. The project is not externally funded.

Who can I contact for further information?

If you have any questions about the study, please contact:

Priyansh Nayak

MSc Computer Science - AVR

If you have concerns about this study and wish to contact someone independent of the research team, you may contact the relevant Trinity College Dublin research ethics process through the School of Computer Science & Statistics.

Appendix C

Informed Consent Form

This appendix reproduces the informed consent form used during live data collection.

Researcher: Priyansh Nayak

Supervisor: Gareth Young

School: School of Computer Science & Statistics, Trinity College Dublin

Please initial each box to confirm your agreement.

1. I confirm that I have read and understood the Participant Information Leaflet for this study, which was provided to me in advance of participation.

Participant initials: _____

2. I have had the opportunity to consider the information provided, ask questions, and have had these questions answered satisfactorily before agreeing to take part.

Participant initials: _____

3. I understand that my participation is voluntary and that I am free to withdraw at any time, without giving a reason and without penalty.

Participant initials: _____

4. I understand that I may stop the session immediately if I experience any discomfort while using the VR headset.

Participant initials: _____

5. I understand that personal data collected for study administration will be stored separately from the research data.

Participant initials: _____

6. I understand that the study uses a Meta VR headset provided to the researcher by the School of Computer Science & Statistics and that the research team will not intentionally collect Meta account data as part of the study.

Participant initials: _____

7. I understand that the research responses will be anonymised for analysis and reporting.

Participant initials: _____

8. I understand that, once my data has been anonymised and included in the analysis, it may no longer be possible to withdraw it.

Participant initials: _____

9. I agree to take part in this study.

Participant initials: _____

Participant Name: _____

Participant Signature: _____

Date: _____

Researcher Name: Priyansh Nayak

Researcher Signature: _____

Date: _____

Appendix D

Recruitment Materials

This appendix reproduces a representative email invitation used during participant recruitment.

Sample Recruitment Email

Subject: Research Study Participation – Creating Novel Virtual Reality Interfaces for Teaching Computer Architecture

Hello,

I am currently recruiting participants for my MSc research study exploring the use of virtual reality for teaching computer architecture concepts.

The study involves attending a single session of approximately 60 minutes, during which participants will use a prototype VR learning environment to complete a small number of guided learning tasks, answer short questionnaires about understanding, engagement, and usability, attempt a knowledge check, and provide brief written feedback on their experience.

Participation is entirely voluntary, and you may withdraw from the study at any time without penalty. During the session, you will be encouraged to take breaks as needed during the VR portion. If you experience any discomfort while using the headset, the session can be stopped immediately.

If you would like to take part, please reply to this email, and you will be provided with further information. Thank you for your time.

Best regards,

Priyansh Nayak